

SHRISTI- ARTIST BOOKING PLATFORM

PROJECT REPORT

A Dissertation Report Submitted in fulfillment of the
requirements for the award of the degree of

“BACHELOR OF COMPUTER APPLICATION”

**UNDER
BENGALURU NORTH UNIVERSITY, KARNATAKA**



FOR THE ACADEMIC YEAR 2025-2026

SUBMITTED BY:

NAYANENDHU C U : (U19CU23S0319

**UNDER THE GUIDENCE OF
Ms. PRAJINA VK**

**Krupanidhi Degree College
Department of Computer Science**



**Krupanidhi Degree College
12/1, ChikkaBellandur, Carmelaram Post
VarthurHobli,Off Sarjapur Road, Bengaluru, Karnataka 560035**

**KRUPANIDHI DEGREE COLLEGE
NO: 12/1, CHIKKABELLANDUR, CARMELARAM POST
VARTHUR (H), BENGALURU-35
(RECOGNISED BY THE BENGALURU NORTH UNIVERSITY)**



CERTIFICATE

This is to certify that major project work entitled “**SHRISTI- ARTIST BOOKING PLATFORM**” is a bonafide work carried out **Ms. Nayanendhu C U**, bearing Reg. No. **(U19CU23S0319)** in fulfillment for the award of **Bachelor of Computer Applications of Bengaluru North University**, during the year 2025-2026. The project report has been approved as it satisfies the academic requirements in respect of Project Work prescribed for the said Degree.

DR. BADARUNNISA S

DECLARATION

I hereby declare that “**SHRISTI- ARTIST BOOKING PLATFORM**” is the result of the project work carried out by me under the guidance of **PRAJINA VK**, Assistant Professor in fulfilment for the award of Bachelor of Computer Applications of Bengaluru North University.

I also declare this project is the outcome of my own efforts and that it has not been submitted to any other University or Institution for the award of any other degree or diploma or certificate.

Place: Bengaluru

Name: NAYANENDHU C U

Date:

CERTIFICATE

This is to certify that the Project report titled “**SHRISTI- ARTIST BOOKING PLATFORM**” is an original work **Mr. NAYANENDHU C U** bearing University Reg no. **U19CU23S0319** and is being submitted in fulfilment for the award of Bachelor of Computer Applications of Bengaluru North University.

Place: Bengaluru

Date:

Guide Signature:

ACKNOWLEDGEMENT

I would like to thank all those who have guided and encouraged me to do what has been done thus far. I avail the opportunity to express our deep sense of gratitude and sincere thanks to our department of Bachelor of Computer Applications.

I am deeply grateful to **Dr. Suresh Nagpal**, Chairman of the KRUPANIDHI GROUP OF INSTITUTIONS. I acknowledge my delightful thanks to **Ms. Geetha Nagpal**, Vice Chair Person, KRUPANIDHI GROUP OF INSTITUTIONS.

I would like to sincerely thank our Dean, **Dr BADRUNNISA S**, Principal, KRUPANIDHI DEGREE COLLEGE for providing the facilities.

I express deep gratitude to my guide **Ms. PRAJINA VK** for her valuable advice, timely suggestions and assistance in the realization of this major-project.

My heartfelt thanks to all the staff members and non-teaching staff of the Computer Science department for providing the necessary facilities and support. Without the support of any one of them, this project would not have been a reality.

I am also grateful to my friends for their valuable suggestions for the success of the project.

I also thank my parents for their support in all our activities, without which I would not have been able to carry out the project.

Place: Bengaluru

Name: NAYANENDHU C U

ABSTRACT

Shristi is a web-based artist discovery and booking platform designed to connect clients with talented artists for custom artwork commissions. The system allows users to browse artists, view their profiles, explore portfolio artworks, and book artists based on style, location, price, and availability. It provides a simple and structured way for clients to find suitable artists while helping artists showcase their work and receive commission requests online.

The platform includes key features such as artist listing, gallery viewing, artist profile pages, portfolio display, booking management, user authentication, artist dashboard, and contact support. Users can explore artworks in the gallery and visit the related artist profile before making a booking, which helps them understand the artist's style and previous work. Artists can present their profile information, upload portfolio images, and manage their presence through the dashboard.

Shristi aims to reduce the gap between artists and customers by providing a centralized digital platform for art discovery and commission booking. The project improves accessibility, saves time for users searching for artists, and creates better visibility for artists. Overall, the system supports a smoother, more transparent, and user-friendly process for discovering, evaluating, and booking artists for personalized artwork.

CONTENTS

Contents

1.INTRODUCTION.....	1
2. MODULES.....	3
3. SOFTWARE REQUIREMENT SPECIFICATION	10
4. SYSTEM ANALYSIS	16
EXISTING SYSTEM:.....	16
5. SYSTEM DESIGN	18
6. DATA FLOW DIAGRAM	20
7. SAMPLE TEST.....	24
8. DATABASE.....	26
9. CODING	27
PYTHON DJANGO CODES.....	27
HOME PAGE.....	86
REGISTER PAGE	86
BROWSE ARTISTS	91
MESSAGES	92
13. FUTURE ENHANCEMENTS.....	98
14. BIBLIOGRAPHY	99

1. INTRODUCTION

Project Name: Srishti - Artist Commission & Booking Platform

Overview:

Srishti is a full-stack web application designed to bridge the gap between talented artists and clients seeking custom artwork. The platform streamlines the process of discovering artists, placing custom art commissions, managing projects, and facilitating secure payments - all in a single, user-friendly ecosystem.

Problem Statement:

Traditionally, artists and clients struggle to connect effectively. Artists lack a centralized platform to showcase their work and manage inquiries, while clients find it difficult to discover talented artists, communicate requirements, and track project progress. This inefficiency leads to miscommunication, delayed projects, and lack of professionalism in transactions.

Solution:

Srishti provides an integrated platform where:

- Artists can build professional profiles, showcase portfolios, set pricing, and manage multiple client commissions efficiently
- Clients can browse artists by art style and location, place bookings with detailed specifications, and track project progress seamlessly
- Transactions are transparent with structured booking workflows, payment verification, and service tier options (Standard, Express, Premium)
- Communication happens in real-time through an integrated messaging system

Key Features:

1. User Management - Role-based authentication (Artist/Client)

2. Artist Profiles - Showcase bio, portfolio, styles, experience, and pricing
3. Booking System - Service request management with status tracking
4. Payment Management - Secure payment verification and advance amount tracking
5. Messaging System - Real-time communication between artists and clients
6. Portfolio Management - Artists can upload and manage artwork samples
7. Location-Based Discovery - Find artists across major Indian cities

Technology Stack:

- Backend: Django (Python) with Django REST Framework
- Database: SQLite (Development) with migration support
- Authentication: JWT with Django REST Framework
- Frontend: HTML5, CSS3, JavaScript
- Media Management: Image upload for portfolios, references, and payment proofs

2. MODULES

This project consists of three Modules:

- Users
- Admin
- Client Users

User Registration:

Users can register themselves on the Srishti platform to book artwork commissions.

Users navigate to the registration page from the website's homepage or login page.

- Full Name: Input field for the user's full name.
- Email Address: Input field for the user's email address. This will be used for account verification and communication.
- Phone Number: Input field for the user's contact number (optional but recommended for better communication).
- Password: Secure password input with strength requirements.

Browse Artists:

Display list of available artists with details such as artist name, art style, location, experience, base price, rating, and availability.

Filter and Search: Allow users to filter artists based on criteria like art style (Portrait, Abstract, Watercolour, Oil Painting, Sketch, Digital, Mandala, Mural), location (Kerala, Delhi, Bengaluru, Mumbai, Chennai, Kolkata, Jaipur, Ahmedabad, Pune), and price range.

Artist Details: Clicking on an artist shows detailed information including bio, portfolio gallery, experience years, mediums used, reviews, and availability calendar.

Place Booking:

It provides a form for clients to submit booking requests with fields for:

- Artist Selection: Pre-populated based on selected artist.
- Artwork Description: Text field for detailed requirements.
- Service Type: Dropdown (Standard, Express, Premium).
- Artwork Size: Input field for dimensions.
- Delivery Type: Options (Digital, Physical, Both).
- Preferred Delivery Date: Date picker for completion date.
- Reference Images: Upload reference images for inspiration.
- Total Price: Automatically calculated based on service type.

Booking Status Tracking:

View current status of each booking: Pending, Accepted, In Progress, Completed, Cancelled, Declined.

Real-Time Messaging:

Clients can communicate with artists through an integrated messaging system with text, file attachments, timestamps, and message history.

Payment Management:

Provide a form for clients to submit payment proof with fields for:

- Payment Screenshot: Upload proof of payment transfer.
- Payment Status: Tracking (Not Requested, Pending Upload, Submitted, Verified).
- Advance Amount Display: Show required advance payment.
- Total Price Breakdown: Itemized pricing by service type.

Forgot Password (Client Can Recover Own Password):

Users navigate to the "Forgot Password" page from the login page.

Forgot Password Form:

- Email Address: Input field for the user to enter their registered email address.
- Submit Button: User clicks on the "Submit" button to initiate the password reset process.
- Reset Link: Verification email sent with secure password reset link.

Change Password:

Provide a link within the user's profile settings or account management section labeled "Change Password."

- Current Password: Input field for the user to enter their current password.
- New Password: Input field for the user to enter their desired new password. Implement password strength requirements for security.
- Confirm New Password: Input field for the user to re-enter the new password to confirm it.
- Update Button: Save new password securely.

Artist Users

Artist Registration & Onboarding:

Artists can register and create professional profiles on the Srishti platform.

Artists navigate to the registration page from the website's homepage or login page.

- Full Name: Input field for the artist's full name.
- Email Address: Input field for the artist's email address. This will be used for account verification and communication.
- Phone Number: Input field for the artist's contact number (recommended).
- Password: Secure password input with strength requirements.

Artist Onboarding Form:

- Bio: Text area for artist biography and description.

- Art Styles: Multi-select from available styles (Portrait, Abstract, Watercolour, Oil Painting, Sketch, Digital, Mandala, Mural).
- Location: Select from major Indian cities (Kerala, Delhi, Bengaluru, Mumbai, Chennai, Kolkata, Jaipur, Ahmedabad, Pune).
- Experience: Input field showing years of experience.
- Base Price: Set minimum pricing for commissions.
- Mediums: Input field for art materials/mediums used.
- Phone: Verified contact number.
- Portfolio URL: Link to external portfolio if available.

Manage Portfolio:

It provides portfolio management with fields for:

- Upload Images: Add artwork images with title and description.
- Image Management: View, edit, delete portfolio images.
- Upload Date: Timestamp automatically recorded.
- Gallery Display: Portfolio visible on artist profile for all users.

Manage Bookings:

Artists navigate to the booking management section from the artist dashboard.

List View: Display a list of all booking requests with key details like date, client name, artwork description, and status.

Filter and Search: Provide options to filter bookings by date, status (pending, accepted, declined, in_progress, completed), client name, or service type.

View Details: Artists can click on a booking to view detailed information including client requirements, reference images, and preferred delivery date.

Update Booking Status:

- Accept Booking: Confirm commission and start project.
- Decline Booking: Reject booking with optional reason.
- Mark In Progress: Update when work begins on artwork.
- Mark Completed: Update when artwork is ready for delivery.
- Add Progress Notes: Include details and milestones.

Real-Time Messaging:

Artists can communicate with clients through integrated messaging system with text, file attachments, timestamps, and full message history.

Manage Profile & Portfolio Settings:

Provide a link within the artist's profile settings section.

- Edit Bio: Update artist biography.
- Edit Art Styles: Modify art styles offered.
- Edit Location: Update service location.
- Edit Experience: Update years of experience.
- Edit Base Price: Modify minimum pricing.
- Edit Mediums: Update art materials used.
- Availability Status: Toggle available/not available.
- Portfolio Management: Add, edit, delete portfolio images.

View Performance Metrics:

- Rating: Display average rating (out of 5 stars).
- Total Bookings: Show completed commissions count.
- Completion Rate: Track booking completion percentage.
- Client Reviews: Display feedback from previous clients.

Admin

Admin Can Create Package (Art Commission Packages):

Admin navigates to the package management section from the admin dashboard.

Manage Packages (Create, Update):

- Create Packages: Admin can create new service tiers including Standard, Express, Premium with pricing and features.
- Edit/Update Packages: Modify existing service packages based on new information or updates.
- Delete Packages: Remove outdated or discontinued packages from the system.

Manage Users:

User Registration & Login: Manage user credentials, roles, and permissions. Admin can create, update, and delete user accounts.

- User Profiles: View and edit user profiles, track user activity and registration history.
- User List: Display all registered users with details like name, email, role (Client/Artist), and registration date.

Manage Enquiries:

Admin navigates to the enquiry management section from the admin dashboard.

List View: Display a list of all enquiries with key details like date, customer name, subject, and status.

Filter and Search: Provide options to filter enquiries by date, status (new, in-progress, resolved), customer name, or category.

Manage Bookings:

Admin navigates to the booking management section from the admin dashboard.

List View: Display a list of all bookings with key details like booking date, client name, artist name, service type, and status.

Filter and Search: Provide options to filter bookings by date, status (pending, accepted, in_progress, completed, cancelled, declined), client name, artist name, or service type.

View Details: Admin can click on a booking to view detailed information including artwork requirements, reference images, payment status, and communication history.

Payment Verification:

Admin navigates to the payment verification section.

- List View: Display all pending payment verifications with booking ID, client, artist, amount, and proof.
- Verification Process: Review payment screenshot and mark as Verified or Disputed.
- Update Status: Change payment status from Pending Upload to Submitted to Verified.
- Add Notes: Document verification details and any issues found.

Manage Issues & Support:

Admin navigates to the issue/support management section from the admin dashboard.

List View: Display a list of all reported issues with key details like date, customer name, category, and status.

Filter and Search: Provide options to filter issues by date, status (new, in-progress, resolved), customer name, or category.

Issue Resolution: Admin can respond to issues, add resolution notes, and update status to resolved.

3. SOFTWARE REQUIREMENT SPECIFICATION

Software Requirements Specifications (SRS) is a document that describes what the software will do and how it will be expected to perform. A software requirements specification (SRS) is a description of a software system to be developed. It lays out functional and non-functional requirements and may include a set of use cases that describe user interactions that the software must provide.

Software Requirement

Name	Details
Operating System	Windows 10
Database Server	SQLITE
Front End	HTML, CSS, Bootstrap,vanilla Js
Back End	Django, JavaScript
Framework	DJango
Application Server	Apache
Browser	Google Chrome, Microsoft Edge

HARDWARE REQUIREMENT

Name	Details
Processor	6th Gen Intel(R) Core (TM) i5
Random-access memory	16.00 GB
SSD	512 GB
Network	Local area network

PROCESSOR:

A processor (CPU) is the logic circuitry that responds to and processes the basic instructions that drive a computer. The CPU is seen as the main and most crucial IC chip in a computer, as it is responsible for interpreting most of computer's commands. Intel® Core™ i5-7200U CPU @

2.60GHz × 4 processor is used in the development of this project.

RANDOM ACCESS MEMORY:

Random-access memory is a form of computer memory that can be read and changed in any order, typically used to store working data and machine code. Project was built with PC of memory space of 16 GB.

OPERATING SYSTEM:

An operating system (OS) is system software that manages computer hardware, software resources, and provides common services for computer programs. It is the program that, after being initially loaded into the computer by a boot program, manages all of the other

application programs in a computer. The application programs make use of the operating system by making requests for services through a defined application program interface (API).

DJANGO:

Django is the main backend framework used in our Shristi project. It helps manage the server-side logic, database, URL routing, user authentication, and API creation. In this project, Django is used to connect the frontend pages like `chat.html`, `artist.html`, `booking.html`, and `login.html` with backend features such as user registration, artist profiles, bookings, portfolio uploads, messaging, and media file handling.

We used **Django 6.0.4** with **Django REST Framework** to build API endpoints for users, artists, bookings, and messages. The project uses a custom user model with roles like `client` and `artist`, allowing different users to access different features. SQLite is used as the database during development, and Django handles uploaded files such as artist avatars, portfolio images, reference images, payment screenshots, and message attachments.

In simple words, Django acts as the backbone of Shristi. It manages data, security, authentication, APIs, and communication between the website interface and the database.

HTML:

HTML (Hypertext Markup Language) is the set of markup symbols or codes inserted in a file intended for display on a World Wide Web browser page. The markup tells the Web browser how to display a Web page's words and images for the user. Each individual markup code is referred to as an element (but many people also refer to it as a tag). Some elements come in pairs that indicate when some display effect is to begin and when it is to end.

HTML is used in our **Shristi** project to create the basic structure of the web pages. It defines the content and layout elements such as headings, paragraphs, buttons, forms, images, navigation bars, cards, and chat sections. In our project, HTML was used to build pages like the home page, artist page, booking page, login page, gallery page, and chat page. It works together with CSS, JavaScript, Bootstrap, and Django templates to display dynamic content and create a user-friendly interface. Overall, HTML forms the foundation of the Shristi website by organizing all the visible content on each page.

CSS:

Cascading Style Sheets (CSS) are a collection of rules we use to define and modify web pages. CSS are similar to styles in Word. CSS allow Web designers to have much more control over their pages look and layout. For instance, you could create a style that defines the body text to be Verdana, 10 points. Later on, you may easily change the body text to Times New Roman, 12 points by just changing the rule in the CSS. Instead of having to change the font on each page of your website, all you need to do is redefine the style on the style sheet, and it will instantly change on all of the pages that the style sheet has been applied to. With HTML styles, the font change would be applied to each instance of that font and have to be changed in each spot. CSS can control the placement of text and objects on your pages as well as the look of those objects.

CSS is used in our **Shristi** project to design and style the web pages. It helps control the layout, colors, fonts, spacing, buttons, navigation bar, cards, forms, and overall appearance of the website. In our project, CSS was used along with Bootstrap to make pages like the home page, artist page, booking page, login page, and chat page look attractive and user-friendly. It also helps improve responsiveness, so the website can adjust properly on different screen sizes such as mobile phones, tablets, and desktops. Overall, CSS made the Shristi website visually appealing, organized, and easy to use.

JAVASCRIPT:

JavaScript is a programming language commonly used in web development. It was originally developed by Netscape as a means to add dynamic and interactive elements to websites. While JavaScript is influenced by Java, the syntax is more similar to C and is based on ECMAScript, a scripting language developed by Sun Microsystems. JavaScript is a client-side scripting language, which means the source code is processed by the client's web browser rather than on the web server.

JavaScript is used in our **Shristi** project to add interactivity and dynamic behavior to the website. It helps handle user actions such as button clicks, form validation, page updates, chat interactions, and API communication with the backend. In our project, JavaScript makes the pages more responsive and functional by allowing users to interact smoothly with features like artist filtering, booking forms, login/register actions, and messaging. Overall, JavaScript improves the user experience by making the website more active and interactive.

BOOTSTRAP:

Bootstrap is a free an open-source front-end web for designing websites and web applications. It contains HTML and CSS-based design templates for forms, buttons, navigations and other interface components, as well as optional JavaScript extensions.

Bootstrap is used in our **Shristi** project to create a clean, responsive, and modern design quickly. It provides ready-made components such as navigation bars, buttons, forms, grids, cards, and responsive layouts. In our project, Bootstrap helped design pages like the home page, artist page, booking page, login page, and chat page in a neat and mobile-friendly way. It also reduces the amount of custom CSS needed and ensures that the website looks good on different devices. Overall, Bootstrap made the Shristi website more organized, responsive, and visually consistent.

DATA BASE:

A database is an organized collection of structured information, or data, typically stored electronically in a computer system. A database is an organized collection of data stored and accessed electronically. Small databases can be stored on a file system, while large databases are hosted on computer clusters or cloud storage.

The database used in our **Shristi** project is SQLite. SQLite is a lightweight database that is easy to set up and suitable for development and academic projects. It stores important project data such as user details, artist profiles, portfolio images, booking information, chat messages, payment details, and uploaded media references. Django connects with SQLite through its built-in ORM, which makes it easier to create, update, retrieve, and delete data from the database.

FRONT END:

The frontend of our **Shristi** project is designed using HTML, CSS, JavaScript, and Bootstrap. It is responsible for the visual part of the website that users interact with. Pages such as the home page, artist page, booking page, login page, gallery page, and chat page are created on the frontend. Bootstrap is used to make the design responsive, while CSS is used for custom styling. JavaScript adds interactivity such as form handling, button actions, filtering, and communication with the backend.

BACK END:

The backend of our Shristi project is developed using Django and Django REST Framework. It manages the main logic of the application, including user registration, login, artist profile management, booking requests, portfolio uploads, and messaging. Django handles URL routing, authentication, API creation, media file handling, and communication between the frontend and database. The backend ensures that user data and project features work securely and properly.

4. SYSTEM ANALYSIS

PROJECT SCOPE:

The scope of the **Shristi** project includes developing a web-based platform that connects clients with artists for customized artwork services. The system allows users to register either as clients or artists. Clients can view artists, explore portfolios, make booking requests, upload reference images, and communicate with artists through the chat feature. Artists can create profiles, upload portfolio images, manage booking requests, and interact with clients.

The project includes frontend pages such as home, artist listing, booking, gallery, login, artist dashboard, artist profile, and chat pages. The backend is developed using Django and Django REST Framework, while SQLite is used as the database to store user details, artist profiles, bookings, messages, and uploaded media files.

EXISTING SYSTEM:

In the existing system, clients usually find artists through social media, personal contacts, or offline references. Communication is mostly done through phone calls, messages, or social media platforms. Booking details, payment information, reference images, and project discussions are often not stored in one organized place. This can create confusion between clients and artists.

Artists also face difficulty in displaying their portfolios, managing bookings, tracking client requirements, and maintaining proper communication. Since most of the work is handled manually, the process becomes time-consuming and less efficient.

- Poor management of artist and client details
- Lack of a centralized booking system
- Difficulty in managing portfolio and artwork references

- Limited communication between client and artist
- Less organized payment and booking tracking

PROPOSED SYSTEM:

The proposed **Shristi** system provides a centralized and user-friendly web platform for clients and artists. It allows clients to search for artists, view their profiles and portfolios, book custom artwork services, upload reference images, and chat directly with artists. Artists can manage their profiles, display their artworks, receive booking requests, and communicate with clients through the system.

The system improves the overall workflow by storing all important details in one place. It reduces manual effort, improves communication, and makes the process of booking custom artwork easier and more organized.

PURPOSE:

The main purpose of the **Shristi** project is to create an online platform that makes it easier for clients to find and book artists for customized artwork. It also helps artists promote their work, manage client requests, and communicate professionally. The system aims to provide a simple, efficient, and organized solution for artist-client interaction, booking management, portfolio display, and messaging.

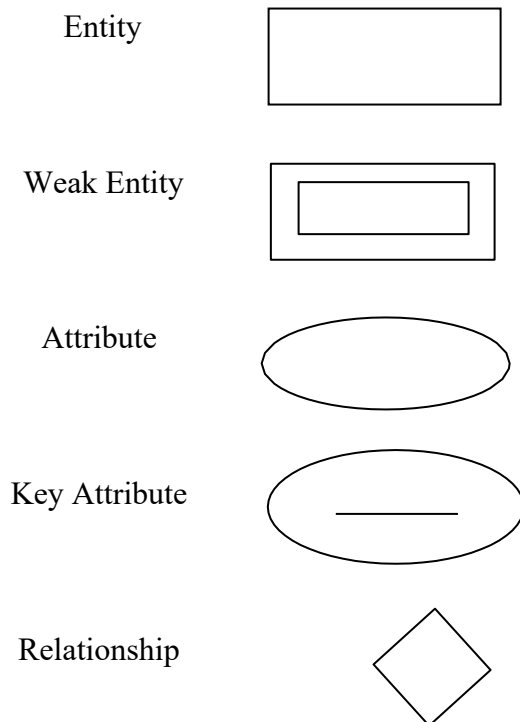
5. SYSTEM DESIGN

It is a process of planning a new business system or replacing an existing system by defining its components or modules to satisfy the specific requirements.

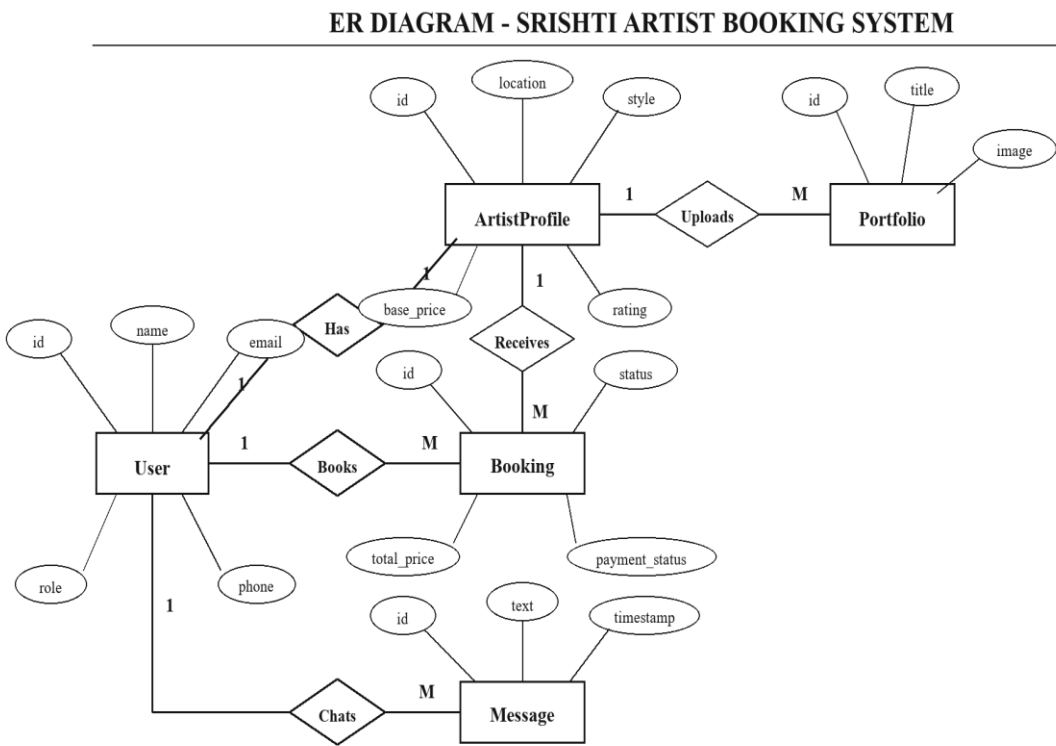
ER DIAGRAM

An Entity Relationship diagram (ERD) show the Relationships of entity sets stored in a database. An entity in this context is a component of data. In other words, ER diagram illustrate the logical structure of Database. At first glance an entity Relationship diagram looks very much like a flowchart. It has the specialized symbols, and the meaning of those symbols, that make it unique.

NOTATIONS FOR ER DIAGRAM



ER DIAGRAM



Important entities and relationships only: User, ArtistProfile, Booking, Portfolio, Message

6. DATA FLOW DIAGRAM

A data flow diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination.

A data flow diagram shows the way information flows through a process or system. It includes data inputs and outputs, data stores, and the various subprocesses the data moves through. DFDs are built using standardized symbols and notation to describe various entities and their relationship.

FEATURES OF DATAFLOW DIAGRAM

A data flow diagram shows the way information flows through a process or system. It includes data inputs and outputs, data stores, and the various subprocesses the data moves through. DFDs are built using standardized symbols and notation to describe various entities and their relationships.

TYPES OF DATA FLOW DIAGRAM:

- Current physical
- Current logical
- New logical
- New physical

CURRENT PHYSICAL

In this DFD process level includes the name of the people or their position or the name or the computer system that might provide some of the overall-processing.

CURRENT LOGICAL

The physical aspects of the system are removed as much as possible so that the current system is reduced to its essence to the data and the processors that transform them regardless of actual physical form.

NEW LOGICAL

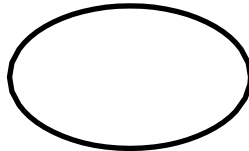
This is actually like a current logical model if the users are happy with the functionality of the current system but had problems with how it was implemented typically through the new logical model.

DFD Symbols

Entity -



Process -



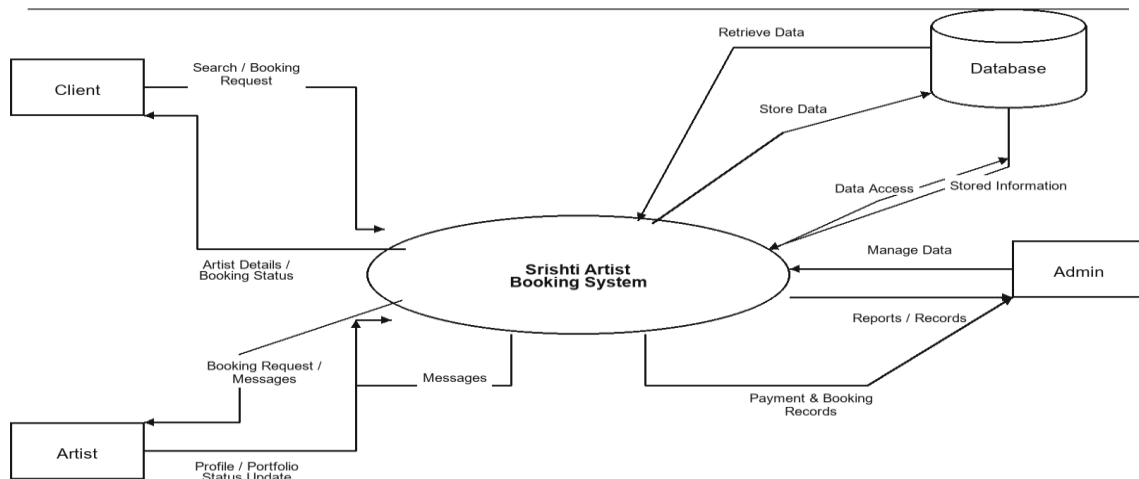
Connector -



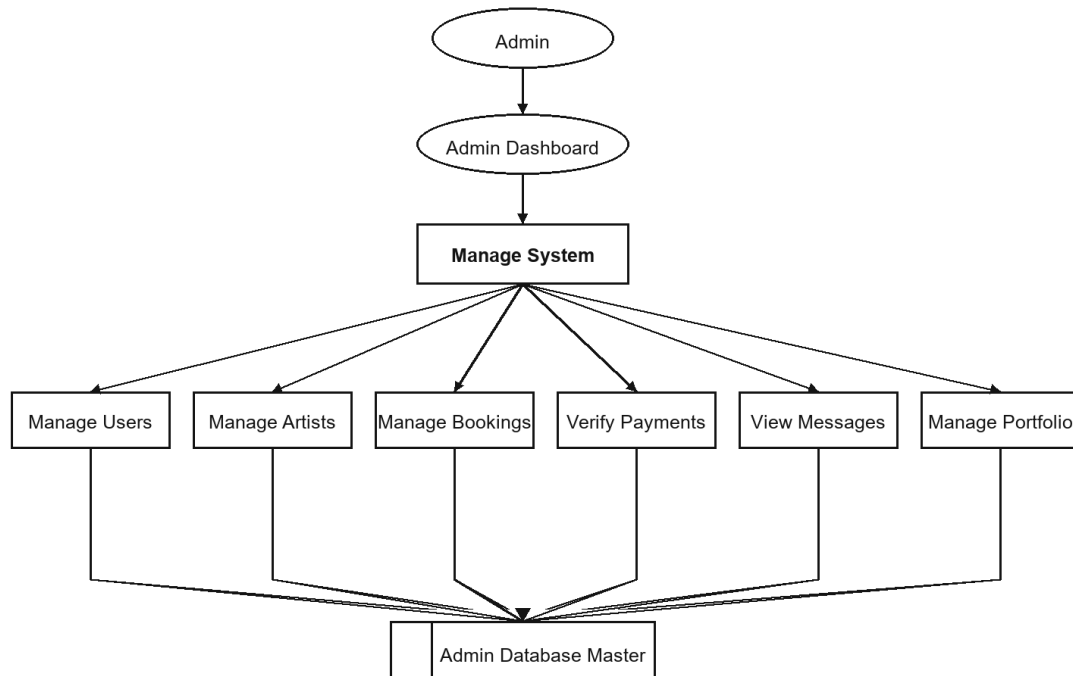
Levels of Data flow diagram:

Level 0:

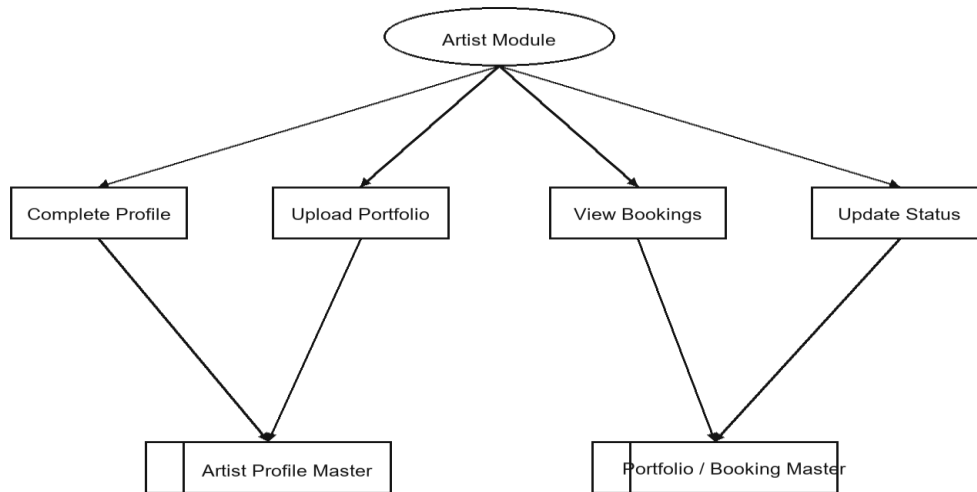
LEVEL 0 DATA FLOW DIAGRAM - SRISHTI ARTIST BOOKING SYSTEM



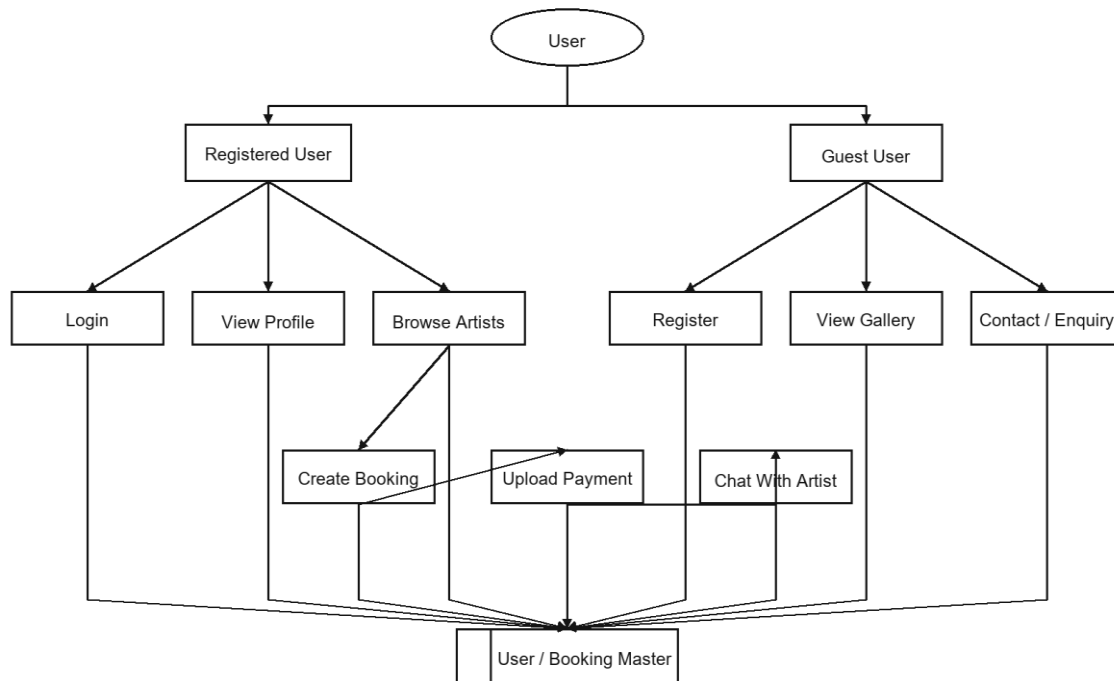
DFD MODULE DIAGRAM - ADMIN LEVEL



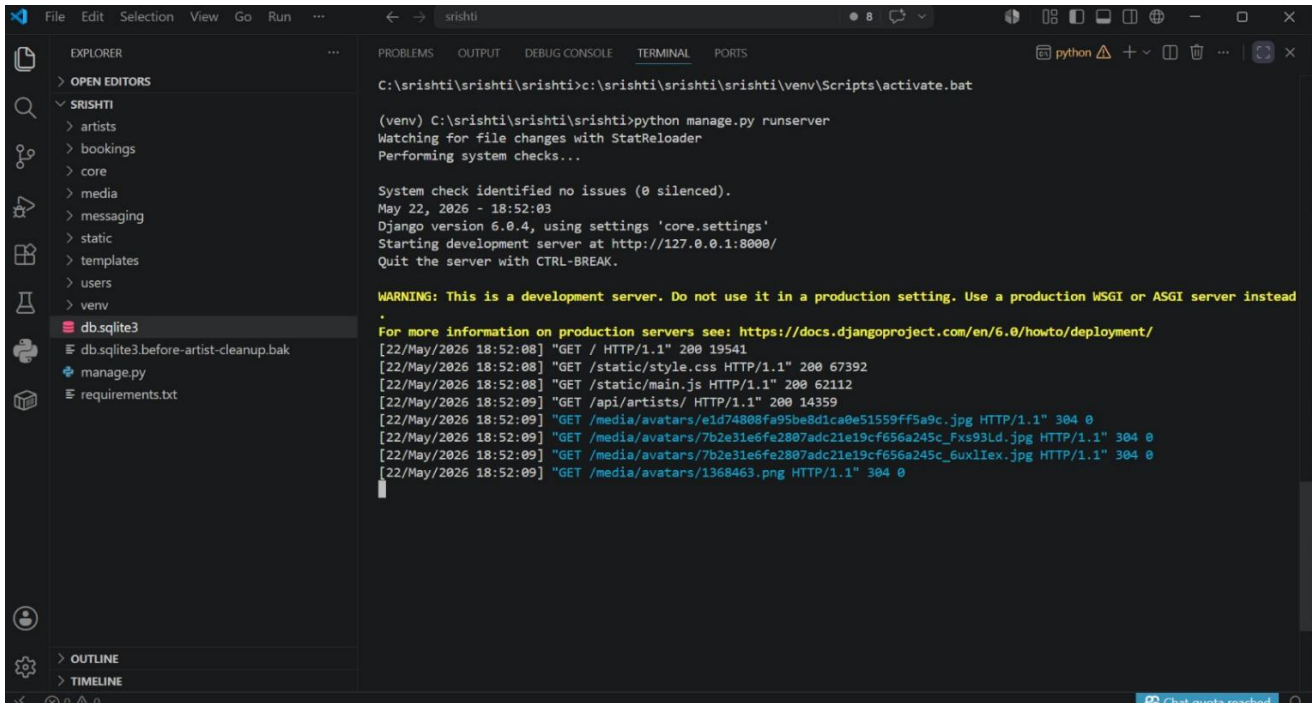
DFD MODULE DIAGRAM - ARTIST / PORTFOLIO LEVEL



DFD MODULE DIAGRAM - USER / CLIENT LEVEL



7. SAMPLE TEST



```
C:\srishti\srishti\srishti>c:\srishti\srishti\srishti\venv\Scripts\activate.bat

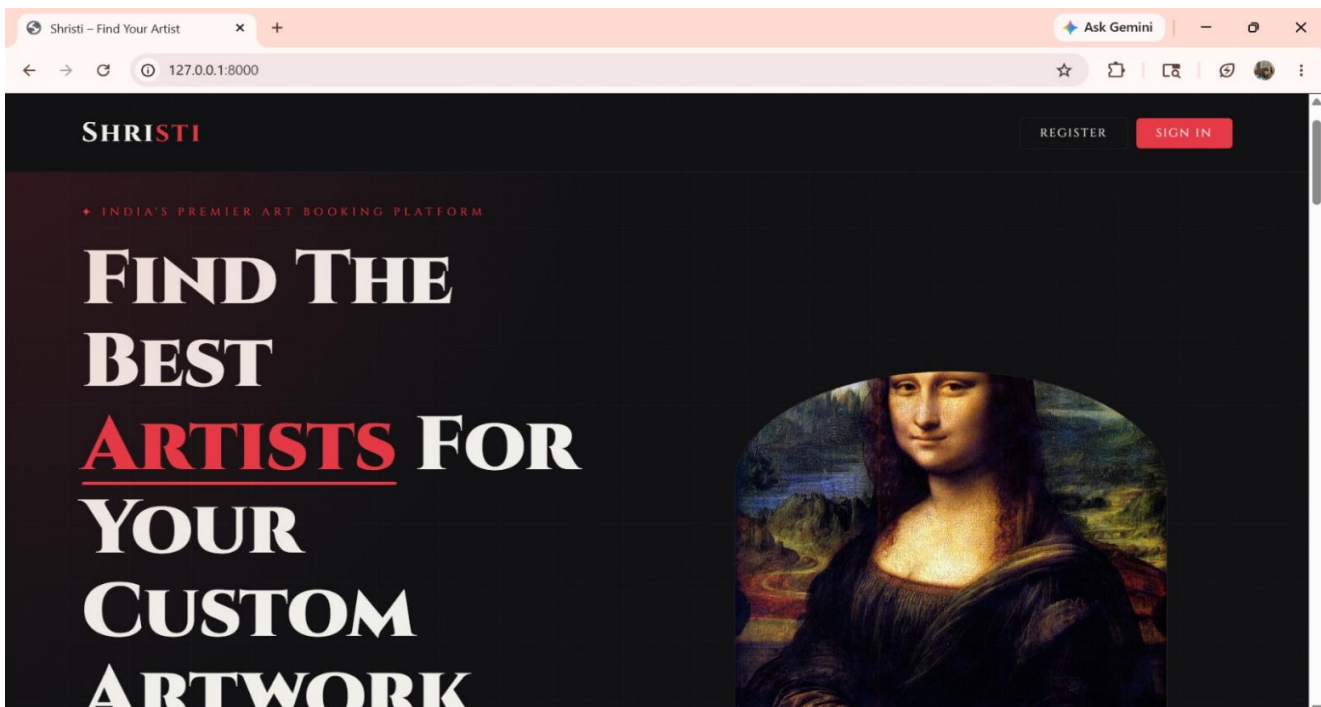
(venv) C:\srishti\srishti\srishti>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
May 22, 2026 - 18:52:03
Django version 6.0.4, using settings 'core.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/

[22/May/2026 18:52:08] "GET / HTTP/1.1" 200 19541
[22/May/2026 18:52:08] "GET /static/style.css HTTP/1.1" 200 67392
[22/May/2026 18:52:08] "GET /static/main.js HTTP/1.1" 200 62112
[22/May/2026 18:52:09] "GET /api/artists/ HTTP/1.1" 200 14359
[22/May/2026 18:52:09] "GET /media/avatars/e1d74808fa95be8d1ca0e51559ff5a9c.jpg HTTP/1.1" 304 0
[22/May/2026 18:52:09] "GET /media/avatars/7b2e31e6fe2807adc21e19cf656a245c_Fxs93Ld.jpg HTTP/1.1" 304 0
[22/May/2026 18:52:09] "GET /media/avatars/7b2e31e6fe2807adc21e19cf656a245c_6ux1Iex.jpg HTTP/1.1" 304 0
[22/May/2026 18:52:09] "GET /media/avatars/1368463.png HTTP/1.1" 304 0
```

The above displayed server is Django development server where it is successfully installed and running properly



The example has been given by a small program of displaying a Shristi Home page in the server.

Templates folder has been created first, then the html pages has been created in the folder of templates, in the html the hello world has been given and the views has been given for that html to return the render.

8. DATABASE

Table 1:ARTIST PROFILE

ID	int(11)	NOT NULL
Bio	Text	NOT NULL
Location	varchar(50)	NOT NULL
Style	varchar(50)	NOT NULL
Base_Price	decimal	NOT NULL
Portfolios_url	varchar(200)	NOT NULL
Experience	Varchar(20)	NOT NULL
Medium	Varchar(255)	NOT NULL
Phone	varchar(20)	NOT NULL

Table 2:USERS _USER

id	int(11)	NOT NULL
Password	varchar(128)	NOT NULL
Username	Varchar (150)	NOT NULL
Name	Varchar(150)	NOT NULL
Phone	varchar(15)	NOT NULL
Email	varchar(254)	NOT NULL

9. CODING

PYTHON DJANGO CODES

USER MODULE

USER MODEL

```
from django.db import models
```

```
from django.contrib.auth.models import AbstractUser
```

```
from django.db import models
```

```
class User(AbstractUser):
```

```
    ROLE_CHOICES = [('client', 'Client'), ('artist', 'Artist')]
```

```
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='client')
```

```
    phone = models.CharField(max_length=15, blank=True)
```

```
    def __str__(self):
```

```
        return f'{self.email} ({self.role})'
```

USER SERIALIZERS

```
from rest_framework import serializers
```

```
from django.contrib.auth import get_user_model
```

```
from artists.models import ArtistProfile
```

```
User = get_user_model()
```

```
class RegisterSerializer(serializers.ModelSerializer):
```

```
    password = serializers.CharField(write_only=True, min_length=6)
```

```
    class Meta:
```

```

model = User
fields = ['id', 'email', 'first_name', 'password', 'role', 'phone']

def create(self, validated_data):
    user = User.objects.create_user(
        username=validated_data['email'],
        email=validated_data['email'],
        password=validated_data['password'],
        first_name=validated_data.get('first_name', ""),
        role=validated_data.get('role', 'client'),
        phone=validated_data.get('phone', "")
    )

    # Auto-create empty artist profile if user is registering as artist
    if user.role == 'artist':
        ArtistProfile.objects.create(
            user=user,
            location='kerala', # Default location
            style='digital'    # Default style
        )

    return user

```

USER VIEWS

```

from django.shortcuts import render

from rest_framework import status
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from django.contrib.auth import get_user_model
from .serializers import RegisterSerializer

```

```

User = get_user_model()

@api_view(['POST'])
def register(request):
    if not request.data.get('terms_accepted'):
        return Response({'terms_accepted': ['You must accept the Terms and Privacy Policy to register.']} , status=400)

    if User.objects.filter(email=request.data.get('email')).exists():
        return Response({'email': ['User already exists.']} , status=400)

    serializer = RegisterSerializer(data=request.data)
    if serializer.is_valid():
        user = serializer.save()
        response_data = {
            'message': 'Account created successfully',
            'user_id': user.id,
            'role': user.role
        }
        # If registered as artist, they need to complete onboarding
        if user.role == 'artist':
            response_data['next_page'] = '/artist-onboarding/'
        else:
            response_data['next_page'] = '/'
        return Response(response_data, status=201)
    return Response(serializer.errors, status=400)

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def me(request):
    user = request.user

```

```
return Response({
    'id': user.id,
    'email': user.email,
    'name': user.get_full_name() or user.username,
    'role': user.role,
    'phone': user.phone
})
```

USER URL ROUTING

```
from django.urls import path
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
from . import views
```

```
urlpatterns = [
    path('register/', views.register),
    path('login/', TokenObtainPairView.as_view()),
    path('token/refresh/', TokenRefreshView.as_view()),
    path('me/', views.me),
]
```

USER ADMIN REGISTRATION

```
from django.contrib import admin
from django.contrib.auth import get_user_model
admin.site.register(get_user_model())
```

ARTIST MODULE

ARTIST MODEL

```
from django.db import models
```

```
from django.conf import settings
```

```
STYLE_CHOICES = [  
    ('portrait', 'Portrait'), ('abstract', 'Abstract'),  
    ('watercolour', 'Watercolour'), ('oil', 'Oil Painting'),  
    ('sketch', 'Sketch'), ('digital', 'Digital'),  
    ('mandala', 'Mandala'), ('mural', 'Mural'),  
]
```

```
LOCATION_CHOICES = [  
    ('kerala', 'Kerala'), ('delhi', 'Delhi'),  
    ('bengaluru', 'Bengaluru'), ('mumbai', 'Mumbai'),  
    ('chennai', 'Chennai'), ('kolkata', 'Kolkata'),  
    ('jaipur', 'Jaipur'), ('ahmedabad', 'Ahmedabad'), ('pune', 'Pune'),  
]
```

```
class ArtistProfile(models.Model):  
    user = models.OneToOneField(settings.AUTH_USER_MODEL,  
on_delete=models.CASCADE, related_name='artist_profile')  
    bio = models.TextField(blank=True)  
    portfolio_url = models.URLField(blank=True)  
    location = models.CharField(max_length=50, choices=LOCATION_CHOICES)  
    style = models.CharField(max_length=50, choices=STYLE_CHOICES)  
    base_price = models.DecimalField(max_digits=10, decimal_places=2, default=0)  
    avatar = models.ImageField(upload_to='avatars/', blank=True, null=True)  
    rating = models.FloatField(default=0.0)  
    total_bookings = models.IntegerField(default=0)  
    is_available = models.BooleanField(default=True)  
    created_at = models.DateTimeField(auto_now_add=True)  
    phone = models.CharField(max_length=20, blank=True)  
    experience = models.CharField(max_length=20, blank=True)  
    mediums = models.CharField(max_length=255, blank=True)
```

```

def __str__(self):
    return f'{self.user.email} - {self.style}'

class Portfolio(models.Model):
    artist = models.ForeignKey(ArtistProfile, on_delete=models.CASCADE,
related_name='portfolio')
    image = models.ImageField(upload_to='portfolio/')
    title = models.CharField(max_length=100, blank=True)
    uploaded_at = models.DateTimeField(auto_now_add=True)

```

ARTIST SERIALIZER

```

from rest_framework import serializers
from .models import ArtistProfile, Portfolio

```

```

class PortfolioSerializer(serializers.ModelSerializer):
    artist_id = serializers.IntegerField(source='artist.id', read_only=True)
    artist_name = serializers.SerializerMethodField()
    artist_style = serializers.CharField(source='artist.style', read_only=True)
    image_url = serializers.SerializerMethodField()

```

```

class Meta:
    model = Portfolio
    fields = [
        'id',
        'title',
        'image',
        'image_url',
        'uploaded_at',
        'artist_id',
        'artist_name',
        'artist_style',

```

```
]
```

```
def get_artist_name(self, obj):  
    return obj.artist.user.get_full_name() or obj.artist.user.email
```

```
def get_image_url(self, obj):  
    request = self.context.get('request')  
    if obj.image:  
        return request.build_absolute_uri(obj.image.url) if request else obj.image.url  
    return None
```

```
class ArtistSerializer(serializers.ModelSerializer):  
    portfolio = PortfolioSerializer(many=True, read_only=True)  
    user_name = serializers.SerializerMethodField()  
    user_id = serializers.IntegerField(source='user.id', read_only=True)  
    avatar_url = serializers.SerializerMethodField()
```

```
class Meta:  
    model = ArtistProfile  
    fields = ['id', 'user', 'user_id', 'user_name', 'bio', 'portfolio_url', 'location', 'style',  
             'base_price', 'avatar', 'avatar_url', 'rating', 'total_bookings',  
             'is_available', 'created_at', 'portfolio', 'phone', 'experience', 'mediums',]  
    read_only_fields = ['user', 'rating', 'total_bookings', 'created_at']
```

```
def get_user_name(self, obj):  
    return obj.user.get_full_name() or obj.user.email
```

```
def get_avatar_url(self, obj):  
    request = self.context.get('request')
```

```

    if obj.avatar:
        return request.build_absolute_uri(obj.avatar.url) if request else obj.avatar.url
    return None

class ArtistOnboardingSerializer(serializers.ModelSerializer):
    """Serializer for artist profile completion during onboarding"""

    user_name = serializers.SerializerMethodField()
    avatar_url = serializers.SerializerMethodField()

    class Meta:
        model = ArtistProfile
        fields = ['user_name', 'bio', 'portfolio_url', 'location', 'style', 'base_price', 'avatar',
'avatar_url', 'phone', 'experience', 'mediums',]
        # Make all fields optional so they can skip them
        extra_kwargs = {
            'bio': {'required': False, 'allow_blank': True},
            'portfolio_url': {'required': False, 'allow_blank': True},
            'location': {'required': False, 'allow_blank': True},
            'style': {'required': False, 'allow_blank': True},
            'base_price': {'required': False},
            'avatar': {'required': False, 'allow_null': True},
            'phone': {'required': False, 'allow_blank': True},
            'experience': {'required': False, 'allow_blank': True},
            'mediums': {'required': False, 'allow_blank': True},

        }

    def get_user_name(self, obj):
        return obj.user.get_full_name() or obj.user.email

    def get_avatar_url(self, obj):

```

```

request = self.context.get('request')
if obj.avatar:
    return request.build_absolute_uri(obj.avatar.url) if request else obj.avatar.url
return None

```

ARTIST VIEWS

```

from django.shortcuts import render

```

```

from rest_framework.decorators import api_view, parser_classes, permission_classes
from rest_framework.parsers import FormParser, JSONParser, MultiPartParser
from rest_framework.permissions import IsAuthenticated, AllowAny
from rest_framework.response import Response
from .models import ArtistProfile, Portfolio
from .serializers import ArtistSerializer, ArtistOnboardingSerializer, PortfolioSerializer

```

```

@api_view(['GET'])
@permission_classes([AllowAny])
def artist_list(request):
    artists = ArtistProfile.objects.filter(is_available=True)

    location = request.GET.get('location')
    style = request.GET.get('style')
    budget = request.GET.get('budget')

    if location and location != 'all':
        artists = artists.filter(location=location)
    if style and style != 'all':
        artists = artists.filter(style=style)
    if budget and budget != 'all':
        budget_map = {
            'under1000': (0, 1000),

```

```

        '1000-5000': (1000, 5000),
        '5000-15000': (5000, 15000),
        '15000-50000': (15000, 50000),
        'above50000': (50000, 9999999),
    }
    low, high = budget_map.get(budget, (0, 9999999))
    artists = artists.filter(base_price__gte=low, base_price__lte=high)

    serializer = ArtistSerializer(artists, many=True, context={'request': request})
    return Response(serializer.data)

@api_view(['GET'])
@permission_classes([AllowAny])
def artist_detail(request, pk):
    try:
        artist = ArtistProfile.objects.get(pk=pk)
        serializer = ArtistSerializer(artist, context={'request': request})
        return Response(serializer.data)
    except ArtistProfile.DoesNotExist:
        return Response({'error': 'Artist not found'}, status=404)

@api_view(['GET', 'POST'])
@permission_classes([AllowAny])
@parser_classes([MultiPartParser, FormParser])
def portfolio_list(request):
    if request.method == 'GET':
        items = Portfolio.objects.select_related('artist', 'artist__user').order_by('-uploaded_at')
        serializer = PortfolioSerializer(items, many=True, context={'request': request})
        return Response(serializer.data)

    if not request.user.is_authenticated:
        return Response({'detail': 'Authentication required.'}, status=401)

```

```

try:
    artist_profile = request.user.artist_profile
except ArtistProfile.DoesNotExist:
    return Response({'detail': 'Only artists can upload portfolio images.'}, status=403)

files = request.FILES.getlist('images') or request.FILES.getlist('image')
if not files:
    return Response({'image': ['No image file was submitted.']], status=400)

created_items = [
    Portfolio.objects.create(
        artist=artist_profile,
        image=file,
        title=request.data.get('title', "")
    )
    for file in files
]
serializer = PortfolioSerializer(created_items, many=True, context={'request': request})
return Response(serializer.data, status=201)

@api_view(['GET', 'PUT'])
@permission_classes([IsAuthenticated])
def update_artist_profile(request):
    profile, created = ArtistProfile.objects.get_or_create(
        user=request.user,
        defaults={'location': "", 'style': "", 'base_price': 0}
    )
    if request.method == 'GET':
        serializer = ArtistSerializer(profile)
        return Response(serializer.data)

```

```

        serializer = ArtistSerializer(profile, data=request.data, partial=True)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=400)

@api_view(['POST', 'GET'])
@permission_classes([IsAuthenticated])
@parser_classes([JSONParser, MultiPartParser, FormParser])
def complete_artist_profile(request):
    """
    GET: Get current artist profile (for onboarding page to fill)
    POST: Update artist profile during onboarding
    """
    try:
        artist_profile = request.user.artist_profile
    except ArtistProfile.DoesNotExist:
        return Response({'error': 'No artist profile found. Please register as artist first.'}, status=404)

    if request.method == 'GET':
        serializer = ArtistOnboardingSerializer(artist_profile, context={'request': request})
        data = dict(serializer.data)
        data['portfolio'] = PortfolioSerializer(
            artist_profile.portfolio.order_by('-uploaded_at'),
            many=True,
            context={'request': request}
        ).data
        return Response(data)

    full_name = request.data.get('full_name')
    if full_name:
        request.user.first_name = full_name

```

```

request.user.save(update_fields=['first_name'])

serializer = ArtistOnboardingSerializer(artist_profile, data=request.data, partial=True)
if serializer.is_valid():
    serializer.save()
    return Response({
        'message': 'Profile completed successfully!',
        'profile': ArtistSerializer(artist_profile, context={'request': request}).data
    }, status=200)
return Response(serializer.errors, status=400)

```

ARTIST URL ROUTING

```

from django.urls import path
from . import views

urlpatterns = [
    path("", views.artist_list),
    path('portfolio/', views.portfolio_list),
    path('<int:pk>', views.artist_detail),
    path('profile/update/', views.update_artist_profile),
    path('profile/onboarding/', views.complete_artist_profile),
]

```

ARTIST ADMIN REGISTRATION

```

from django.contrib import admin
from .models import ArtistProfile, Portfolio
admin.site.register(ArtistProfile)
admin.site.register(Portfolio)

```

BOOKING MODULE

BOOKING MODEL

```
from django.db import models
from django.conf import settings
from artists.models import ArtistProfile
```

```
class Booking(models.Model):
```

```
    STATUS_CHOICES = [
```

```
        ('pending', 'Pending'),
```

```
        ('accepted', 'Accepted'),
```

```
        ('in_progress', 'In Progress'),
```

```
        ('completed', 'Completed'),
```

```
        ('cancelled', 'Cancelled'),
```

```
        ('declined', 'Declined'),
```

```
    ]
```

```
    DELIVERY_CHOICES = [('digital', 'Digital'), ('physical', 'Physical'), ('both', 'Both')]
```

```
    SERVICE_CHOICES = [('standard', 'Standard'), ('express', 'Express'), ('premium', 'Premium')]
```

```
    PAYMENT_CHOICES = [
```

```
        ('not_requested', 'Not Requested'),
```

```
        ('pending_upload', 'Pending Upload'),
```

```
        ('submitted', 'Submitted'),
```

```
        ('verified', 'Verified'),
```

```
    ]
```

```
    client = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name='bookings')
```

```
    artist = models.ForeignKey(ArtistProfile, on_delete=models.CASCADE,
related_name='bookings')
```

```
    description = models.TextField()
```

```
    service_type = models.CharField(max_length=20, choices=SERVICE_CHOICES,
default='standard')
```

```

artwork_size = models.CharField(max_length=50, blank=True)
    delivery = models.CharField(max_length=20, choices=DELIVERY_CHOICES,
default='digital')
    delivery_address = models.TextField(blank=True)
    preferred_date = models.DateField()
    reference_image = models.ImageField(upload_to='references/', blank=True, null=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')
    total_price = models.DecimalField(max_digits=10, decimal_places=2)
    advance_amount = models.DecimalField(max_digits=10, decimal_places=2, default=0)
    payment_status = models.CharField(max_length=20, choices=PAYMENT_CHOICES,
default='not_requested')
    payment_screenshot = models.ImageField(upload_to='payment_screenshots/', blank=True,
null=True)
    payment_uploaded_at = models.DateTimeField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Booking #{self.id} - {self.client.email} → {self.artist}"

```

BOOKING SERIALIZERS

```

from rest_framework import serializers
from .models import Booking

class BookingSerializer(serializers.ModelSerializer):
    artist_name = serializers.SerializerMethodField()
    client_name = serializers.SerializerMethodField()
    reference_image_url = serializers.SerializerMethodField()
    payment_screenshot_url = serializers.SerializerMethodField()

    class Meta:
        model = Booking

```

```

fields = ['id', 'client', 'artist', 'artist_name', 'client_name',
          'description', 'service_type', 'artwork_size',
          'delivery', 'delivery_address', 'preferred_date', 'reference_image',
          'reference_image_url', 'status', 'total_price',
          'advance_amount', 'payment_status', 'payment_screenshot',
          'payment_screenshot_url', 'payment_uploaded_at', 'created_at']
read_only_fields = [
    'client',
    'status',
    'total_price',
    'advance_amount',
    'payment_status',
    'payment_uploaded_at',
    'created_at',
]

def get_artist_name(self, obj):
    return obj.artist.user.get_full_name() or obj.artist.user.email

def get_client_name(self, obj):
    return obj.client.get_full_name() or obj.client.email

def get_reference_image_url(self, obj):
    request = self.context.get('request')
    if not obj.reference_image:
        return None
    return request.build_absolute_uri(obj.reference_image.url) if request else
obj.reference_image.url

def get_payment_screenshot_url(self, obj):
    request = self.context.get('request')
    if not obj.payment_screenshot:

```

```
        return None

    return request.build_absolute_uri(obj.payment_screenshot.url) if request else
obj.payment_screenshot.url
```

BOOKING VIEWS

```
from django.shortcuts import render
```

```
from django.utils import timezone
from rest_framework.decorators import api_view, parser_classes, permission_classes
from rest_framework.parsers import FormParser, JSONParser, MultiPartParser
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from .models import Booking
from .serializers import BookingSerializer
from artists.models import ArtistProfile
from messaging.models import Message
```

```
def notify(sender, receiver, text):
```

```
    Message.objects.create(sender=sender, receiver=receiver, text=text)
```

```
@api_view(['POST'])
```

```
@permission_classes([IsAuthenticated])
```

```
@parser_classes([JSONParser, MultiPartParser, FormParser])
```

```
def create_booking(request):
```

```
    if request.user.role != 'client':
```

```
        return Response({'error': 'Only client accounts can send booking requests.'}, status=403)
```

```
    if request.data.get('terms_accepted') != 'true':
```

```
        return Response({'error': 'You must accept the Terms of Service before booking.'}, status=400)
```

```
    delivery = request.data.get('delivery', 'digital')
```

```

if delivery in ['physical', 'both'] and not request.data.get('delivery_address', '').strip():
    return Response({'error': 'Delivery address is required for physical artwork delivery.'},
status=400)

artist_id = request.data.get('artist')
try:
    artist = ArtistProfile.objects.get(pk=artist_id)
except ArtistProfile.DoesNotExist:
    return Response({'error': 'Artist not found'}, status=404)

if artist.user_id == request.user.id:
    return Response({'error': 'You cannot book your own artist profile.'}, status=400)

multipliers = {'standard': 1, 'express': 1.5, 'premium': 2}
service = request.data.get('service_type', 'standard')
price = float(artist.base_price) * multipliers.get(service, 1)
platform_fee = price * 0.05
total = round(price + platform_fee, 2)
advance = round(total * 0.30, 2)

data = request.data.copy()
data['total_price'] = total

serializer = BookingSerializer(data=data, context={'request': request})
if serializer.is_valid():
    booking = serializer.save(
        client=request.user,
        total_price=total,
        advance_amount=advance,
    )
    notify(
        request.user,

```

```

        artist.user,
        f'New booking request from {request.user.get_full_name() or request.user.email}.'
    )
    return Response(serializer.data, status=201)
    return Response(serializer.errors, status=400)

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def my_bookings(request):
    if request.user.role == 'artist':
        bookings = Booking.objects.filter(artist__user=request.user).exclude(status__in=['declined',
        'cancelled'])
    else:
        bookings = Booking.objects.filter(client=request.user)
    serializer = BookingSerializer(bookings.order_by('-created_at'), many=True, context={'request':
    request})
    return Response(serializer.data)

@api_view(['PATCH'])
@permission_classes([IsAuthenticated])
def update_booking_status(request, pk):
    try:
        booking = Booking.objects.get(pk=pk, artist__user=request.user)
        new_status = request.data.get('status', booking.status)
        allowed_statuses = {'accepted', 'declined', 'in_progress', 'completed', 'cancelled'}
        if new_status not in allowed_statuses:
            return Response({'error': 'Invalid status'}, status=400)

        if new_status == 'accepted':
            booking.status = 'accepted'
            booking.payment_status = 'pending_upload'
            if not booking.advance_amount:

```

```

        booking.advance_amount = round(float(booking.total_price) * 0.30, 2)
    notify(
        request.user,
        booking.client,
        'Your booking was accepted. Please upload the advance payment screenshot to start the
work.'
    )
    elif new_status == 'declined':
        booking.status = 'declined'
        notify(
            request.user,
            booking.client,
            'Your booking was declined by the artist.'
        )
    elif new_status == 'in_progress':
        if booking.payment_status != 'submitted':
            return Response({'error': 'Payment screenshot must be submitted before starting work.'},
status=400)
        booking.status = 'in_progress'
        booking.payment_status = 'verified'
        notify(
            request.user,
            booking.client,
            'Your advance payment was verified. The artist has started working.'
        )
    else:
        booking.status = new_status
        booking.save()
        serializer = BookingSerializer(booking, context={'request': request})
        return Response(serializer.data)
except Booking.DoesNotExist:
    return Response({'error': 'Booking not found'}, status=404)

```

```

@api_view(['POST'])
@permission_classes([IsAuthenticated])
@parser_classes([MultiPartParser, FormParser])
def upload_payment_screenshot(request, pk):
    try:
        booking = Booking.objects.get(pk=pk, client=request.user)
    except Booking.DoesNotExist:
        return Response({'error': 'Booking not found'}, status=404)

    if booking.status != 'accepted':
        return Response({'error': 'Payment screenshot can be uploaded only after artist acceptance.'},
            status=400)

    screenshot = request.FILES.get('payment_screenshot')
    if not screenshot:
        return Response({'error': 'Payment screenshot is required.'}, status=400)
    if not getattr(screenshot, 'content_type', "").startswith('image/'):
        return Response({'error': 'Please upload an image screenshot.'}, status=400)
    if screenshot.size > 10 * 1024 * 1024:
        return Response({'error': 'Screenshot must be 10 MB or smaller.'}, status=400)

    booking.payment_screenshot = screenshot
    booking.payment_status = 'submitted'
    booking.payment_uploaded_at = timezone.now()
    booking.save()
    notify(
        request.user,
        booking.artist.user,
        'Payment screenshot submitted. Please verify it and start the work.'
    )
    serializer = BookingSerializer(booking, context={'request': request})

```

```
return Response(serializer.data)
```

BOOKING URL ROUTING

```
from django.urls import path  
from . import views
```

```
urlpatterns = [  
    path('create/', views.create_booking),  
    path('mine/', views.my_bookings),  
    path('<int:pk>/status/', views.update_booking_status),  
    path('<int:pk>/payment/', views.upload_payment_screenshot),  
]
```

BOOKING ADMIN REGISTRATION

```
from django.contrib import admin  
from .models import Booking  
admin.site.register(Booking)
```

MESSAGE MODULE

MESSAGE MODEL

```
from django.db import models  
from django.conf import settings
```

```
class Message(models.Model):  
    sender = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,  
related_name='sent_messages')  
    receiver = models.ForeignKey(settings.AUTH_USER_MODEL,  
on_delete=models.CASCADE, related_name='received_messages')  
    text = models.TextField(blank=True)  
    attachment = models.FileField(upload_to='message_attachments/', blank=True, null=True)
```

```

attachment_name = models.CharField(max_length=255, blank=True)
attachment_content_type = models.CharField(max_length=120, blank=True)
timestamp = models.DateTimeField(auto_now_add=True)
is_read = models.BooleanField(default=False)

class Meta:
    ordering = ['timestamp']

```

MEESSAGE SERIALIZER

```

from rest_framework import serializers
from .models import Message

class MessageSerializer(serializers.ModelSerializer):
    sender_name = serializers.SerializerMethodField()
    receiver_name = serializers.SerializerMethodField()
    attachment_url = serializers.SerializerMethodField()

class Meta:
    model = Message
    fields = [
        'id',
        'sender',
        'sender_name',
        'receiver',
        'receiver_name',
        'text',
        'attachment',
        'attachment_url',
        'attachment_name',
        'attachment_content_type',
    ]

```

```

        'timestamp',
        'is_read',
    ]
    read_only_fields = ['sender', 'receiver', 'timestamp', 'is_read']

def get_sender_name(self, obj):
    return obj.sender.get_full_name() or obj.sender.email

def get_receiver_name(self, obj):
    return obj.receiver.get_full_name() or obj.receiver.email

def get_attachment_url(self, obj):
    request = self.context.get('request')
    if not obj.attachment:
        return None
    return request.build_absolute_uri(obj.attachment.url) if request else obj.attachment.url

def validate(self, attrs):
    text = attrs.get('text', "")
    attachment = attrs.get('attachment')
    if not text.strip() and not attachment:
        raise serializers.ValidationError('Message text or an attachment is required.')
    if attachment:
        max_size = 10 * 1024 * 1024
        allowed_types = {
            'image/jpeg',
            'image/png',
            'image/gif',
            'image/webp',
            'application/pdf',
            'application/msword',
            'application/vnd.openxmlformats-officedocument.wordprocessingml.document',

```

```

        'text/plain',
    }
    if attachment.size > max_size:
        raise serializers.ValidationError('Attachment must be 10 MB or smaller.')
    if getattr(attachment, 'content_type', '') not in allowed_types:
        raise serializers.ValidationError('Unsupported attachment type.')
    return attrs

```

MESSAGE VIEWS

```

from rest_framework.decorators import api_view, parser_classes, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.parsers import FormParser, JSONParser, MultiPartParser
from rest_framework.response import Response
from django.contrib.auth import get_user_model
from django.db.models import Q, Count
from .models import Message
from .serializers import MessageSerializer
from artists.models import ArtistProfile
from bookings.models import Booking

```

```

User = get_user_model()

```

```

def display_name(user):
    return user.get_full_name() or user.email or user.username

```

```

def artist_profile_for(user):
    try:
        return user.artist_profile
    except ArtistProfile.DoesNotExist:
        return None

```

```

def allowed_contact_ids_for(user):
    message_contact_ids = set(
        Message.objects
        .filter(Q(sender=user) | Q(receiver=user))
        .values_list('sender_id', 'receiver_id')
    )
    message_contact_ids = {
        user_id
        for pair in message_contact_ids
        for user_id in pair
        if user_id != user.id
    }

    if user.role == 'artist':
        booking_contact_ids = set(
            Booking.objects
            .filter(artist__user=user)
            .values_list('client_id', flat=True)
        )
        return booking_contact_ids | message_contact_ids

    booking_contact_ids = set(
        Booking.objects
        .filter(client=user)
        .values_list('artist__user_id', flat=True)
    )
    return booking_contact_ids | message_contact_ids

def users_can_message(user, other_user):
    if user.id == other_user.id:
        return False

```

```

    if other_user.id in allowed_contact_ids_for(user):
        return True

    return user.role == 'client' and other_user.role == 'artist'

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def inbox(request):
    allowed_contact_ids = allowed_contact_ids_for(request.user)
    if not allowed_contact_ids:
        return Response([])

    messages = (
        Message.objects
        .filter(Q(sender=request.user) | Q(receiver=request.user))
        .select_related('sender', 'receiver')
        .order_by('-timestamp')
    )

    latest_by_user = {}
    for message in messages:
        other = message.receiver if message.sender_id == request.user.id else message.sender
        if other.id not in allowed_contact_ids:
            continue
        if other.id not in latest_by_user:
            artist_profile = artist_profile_for(other)
            avatar_url = None
            if artist_profile and artist_profile.avatar:
                avatar_url = request.build_absolute_uri(artist_profile.avatar.url)
            latest_by_user[other.id] = {
                'user_id': other.id,
                'name': display_name(other),

```

```

        'role': other.role,
        'artist_profile_id': artist_profile.id if artist_profile else None,
        'avatar_url': avatar_url,
        'last_message': message.text or (message.attachment_name or 'Attachment'),
        'last_sender_id': message.sender_id,
        'timestamp': message.timestamp,
        'unread_count': 0,
    }

```

```

unread_counts = (
    Message.objects
    .filter(receiver=request.user, sender_id__in=allowed_contact_ids, is_read=False)
    .values('sender')
    .annotate(count=Count('id'))
)

```

```

for item in unread_counts:
    sender_id = item['sender']
    if sender_id in latest_by_user:
        latest_by_user[sender_id]['unread_count'] = item['count']

```

```

return Response(list(latest_by_user.values()))

```

```

@api_view(['GET', 'POST'])
@permission_classes([IsAuthenticated])
@parser_classes([JSONParser, MultiPartParser, FormParser])
def conversation(request, user_id):
    try:
        other_user = User.objects.get(pk=user_id)
    except User.DoesNotExist:
        return Response({'error': 'User not found'}, status=404)

    if not users_can_message(request.user, other_user):

```

```
    return Response({'error': 'This conversation is available only between art lovers and artists.'},
status=403)
```

```
if request.method == 'GET':
    messages = Message.objects.filter(
        sender=request.user, receiver=other_user
    ) | Message.objects.filter(
        sender=other_user, receiver=request.user
    )
    Message.objects.filter(sender=other_user, receiver=request.user,
is_read=False).update(is_read=True)
    serializer = MessageSerializer(messages.order_by('timestamp'), many=True,
context={'request': request})
    return Response(serializer.data)
```

```
if request.method == 'POST':
    serializer = MessageSerializer(data=request.data, context={'request': request})
    if serializer.is_valid():
        attachment = request.FILES.get('attachment')
        save_kwargs = {'sender': request.user, 'receiver': other_user}
        if attachment:
            save_kwargs.update({
                'attachment_name': attachment.name,
                'attachment_content_type': getattr(attachment, 'content_type', "") or "",
            })
        serializer.save(**save_kwargs)
        return Response(serializer.data, status=201)
    return Response(serializer.errors, status=400)
```

MESSAGE URL ROUTING

```
from django.urls import path
```

```
from . import views

urlpatterns = [
    path("", views.inbox),
    path('<int:user_id>', views.conversation),
]
```

MAIN PROJECT CONFIGURATION

DJANGO SETTING

Django settings for core project.

Generated by 'django-admin startproject' using Django 6.0.4.

For more information on this file, see
<https://docs.djangoproject.com/en/6.0/topics/settings/>

For the full list of settings and their values, see
<https://docs.djangoproject.com/en/6.0/ref/settings/>
"""

```
from pathlib import Path
```

```
# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent
```

```
# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/
```

```
# SECURITY WARNING: keep the secret key used in production secret!
```

```
SECRET_KEY = 'django-insecure-uocu=3vs(ef4k=l3)nflki=86d2m8%_e$*%7bu@i53z9d)o69a'
```

```
# SECURITY WARNING: don't run with debug turned on in production!
```

```
DEBUG = True
```

```
ALLOWED_HOSTS = ['localhost', '127.0.0.1', 'testserver']
```

```
# Application definition
```

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'corsheaders',  
    'users',  
    'artists',  
    'bookings',  
    'messaging',  
]
```

```
MIDDLEWARE = [  
    'corsheaders.middleware.CorsMiddleware',  
    'django.middleware.security.SecurityMiddleware',  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    'django.middleware.csrf.CsrfViewMiddleware',  
    'django.contrib.auth.middleware.AuthenticationMiddleware',  
    'django.contrib.messages.middleware.MessageMiddleware',  
]
```

```

'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'core.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]

WSGI_APPLICATION = 'core.wsgi.application'

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

```

```
# Password validation
# https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

```
# Internationalization
# https://docs.djangoproject.com/en/6.0/topics/i18n/
```

```
LANGUAGE_CODE = 'en-us'
```

```
TIME_ZONE = 'UTC'
```

```
USE_I18N = True
```

```
USE_TZ = True
```

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/6.0/howto/static-files/
```

```
STATIC_URL = 'static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / 'static',  
]
```

```
AUTH_USER_MODEL = 'users.User'
```

```
CORS_ALLOW_ALL_ORIGINS = True
```

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': (  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ),  
}
```

```
from datetime import timedelta
```

```
SIMPLE_JWT = {  
    'ACCESS_TOKEN_LIFETIME': timedelta(days=1),  
}
```

```
MEDIA_URL = '/media/'
```

```
MEDIA_ROOT = BASE_DIR / 'media'
```

MAIN URL CONFIGURATION

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
from django.conf import settings
```

```
from django.conf.urls.static import static
```

```
from django.views.generic import TemplateView
```

```

urlpatterns = [
    path('admin/', admin.site.urls),

    # Page routes
    path("", TemplateView.as_view(template_name='index.html'), name='home'),
    path('index/', TemplateView.as_view(template_name='index.html'), name='home2'),
    path('artist/', TemplateView.as_view(template_name='artist.html'), name='artist'),
        path('artist-dashboard/', TemplateView.as_view(template_name='artist-dashboard.html'),
name='artist_dashboard'),
        path('artist-profile/', TemplateView.as_view(template_name='artist-profile.html'),
name='artist_profile'),
    path('booking/', TemplateView.as_view(template_name='booking.html'), name='booking'),
    path('chat/', TemplateView.as_view(template_name='chat.html'), name='chat'),
    path('contact/', TemplateView.as_view(template_name='contact.html'), name='contact'),
        path('forgot-password/', TemplateView.as_view(template_name='forgot-password.html'),
name='forgot_password'),
    path('login/', TemplateView.as_view(template_name='login.html'), name='login'),
        path('privacy-policy/', TemplateView.as_view(template_name='privacy-policy.html'),
name='privacy_policy'),
    path('terms/', TemplateView.as_view(template_name='terms.html'), name='terms'),

        path('artist-onboarding/', TemplateView.as_view(template_name='artist-onboarding.html'),
name='artist_onboarding'),

    path('gallery/', TemplateView.as_view(template_name='gallery.html'), name='gallery'),

    # API routes — each has its own prefix
    path('api/users/', include('users.urls')),
    path('api/artists/', include('artists.urls')),
    path('api/bookings/', include('bookings.urls')),
    path('api/messages/', include('messaging.urls')),

```

```
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

STATIC FILES

STYLE.CSS

```
:root {  
  --brand-red: #e63946;  
  --brand-red-dk: #c1121f;  
  --bg-dark: #111114;  
  --bg-card: #1a1a1f;  
  --text-light: #f1f0eb;  
  --text-muted: #9a9a9a;  
  --border: rgba(255,255,255,0.08);  
  --radius: 14px;  
  --radius-sm: 6px;  
  --font-display: 'Cinzel', serif;  
  --font-body: 'Raleway', sans-serif;  
  --transition: 0.35s cubic-bezier(.25,.8,.25,1);  
}  
  
*, *::before, *::after {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
}  
  
body {  
  font-family: var(--font-body);  
  background: var(--bg-dark);  
  color: var(--text-light);  
  min-height: 100vh;  
  overflow-x: hidden;
```

```
}
```

```
a { text-decoration: none; color: inherit; }
```

```
.btn-primary-red,
```

```
.btn-outline,
```

```
.btn-book {
```

```
  font-family: var(--font-display);
```

```
  letter-spacing: 1.5px;
```

```
  text-transform: uppercase;
```

```
  border-radius: var(--radius-sm);
```

```
  cursor: pointer;
```

```
  transition: all var(--transition);
```

```
}
```

```
.btn-primary-red {
```

```
  background: var(--brand-red);
```

```
  color: #fff;
```

```
  padding: 0.75rem 2rem;
```

```
}
```

```
.btn-primary-red:hover {
```

```
  background: var(--brand-red-dk);
```

```
  transform: translateY(-2px);
```

```
}
```

```
.btn-outline {
```

```
  background: transparent;
```

```
  color: var(--text-light);
```

```
  border: 1px solid var(--border);
```

```
  padding: 0.75rem 2rem;
```

```
}
```

```
.btn-outline:hover {
```

```

border-color: var(--text-light);
color: #fff;
}

#mainNav {
  position: sticky;
  top: 0;
  z-index: 1000;
  background: rgba(17,17,20,0.85);
  border-bottom: 1px solid var(--border);
  backdrop-filter: blur(18px);
}

.navbar-brand {
  font-family: var(--font-display);
  font-size: 1.7rem;
  letter-spacing: 2px;
}

.navbar-brand span { color: var(--brand-red); }

.navbar-nav .nav-link {
  font-family: var(--font-display);
  color: var(--text-muted) !important;
  padding: 0.5rem 1rem !important;
  position: relative;
}

.navbar-nav .nav-link:hover,
.navbar-nav .nav-link.active {
  color: var(--text-light) !important;
}

.navbar-nav .nav-link::after {
  content: " ";

```

```

position: absolute;
bottom: 2px;
left: 1rem;
right: 1rem;
height: 1px;
background: var(--brand-red);
transform: scaleX(0);
transition: transform var(--transition);
}

.navbar-nav .nav-link:hover::after,
.navbar-nav .nav-link.active::after {
  transform: scaleX(1);
}

.section-eyebrow {
  font-family: var(--font-display);
  color: var(--brand-red);
  letter-spacing: 4px;
  text-transform: uppercase;
  margin-bottom: 0.6rem;
}

.section-title {
  font-size: clamp(1.6rem, 3.5vw, 2.6rem);
  font-weight: 700;
  line-height: 1.15;
}

.section-subtitle {
  color: var(--text-muted);
  margin-top: 0.6rem;
  line-height: 1.7;
}

```

```

.reveal {
  opacity: 0;
  transform: translateY(28px);
  transition: opacity 0.65s ease, transform 0.65s ease;
}

.reveal.visible {
  opacity: 1;
  transform: translateY(0);
}

.artist-card {
  background: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  overflow: hidden;
  transition: transform var(--transition), border-color var(--transition), box-shadow var(--transition);
}

.artist-card:hover {
  transform: translateY(-8px);
  border-color: rgba(230,57,70,0.40);
  box-shadow: 0 20px 50px rgba(0,0,0,0.4);
}

.artist-img-wrap {
  position: relative;
  overflow: hidden;
  height: 220px;
}

.artist-img-wrap img {
  width: 100%;
  height: 100%;
}

```

```

    object-fit: cover;
    transition: transform 0.6s ease;
  }
  .artist-card:hover .artist-img-wrap img {
    transform: scale(1.08);
  }

  .filter-control {
    width: 100%;
    background: var(--bg-dark);
    border: 1px solid var(--border);
    color: var(--text-light);
    border-radius: var(--radius-sm);
    padding: 0.58rem 0.9rem;
    font-family: var(--font-body);
    transition: border-color var(--transition), box-shadow var(--transition);
  }
  .filter-control:focus {
    border-color: var(--brand-red);
    box-shadow: 0 0 0 3px rgba(230,57,70,0.15);
  }

```

MAIN .JS

```

"use strict";

const API_BASE = "http://127.0.0.1:8000/api";
const DEFAULT_ARTIST_IMAGE = "https://i.pravatar.cc/400?img=47";

function saveAuth(data) {
  localStorage.setItem("access_token", data.access);
  localStorage.setItem("refresh_token", data.refresh);
}

```

```

function getToken() {
  return localStorage.getItem("access_token");
}

async function authFetch(url, options = {}) {
  const headers = {
    Authorization: `Bearer ${getToken()}`,
    ...(options.headers || {})
  };
  if (!(options.body instanceof FormData) && !headers["Content-Type"]) {
    headers["Content-Type"] = "application/json";
  }
  return fetch(url, { ...options, headers });
}

function clearAuth() {
  localStorage.removeItem("access_token");
  localStorage.removeItem("refresh_token");
  localStorage.removeItem("userRole");
  localStorage.removeItem("userName");
}

async function getCurrentUser() {
  if (!getToken()) return null;
  try {
    const response = await authFetch(`${API_BASE}/users/me/`);
    if (!response.ok) { clearAuth(); return null; }
    const me = await response.json();
    localStorage.setItem("userRole", me.role || "client");
    localStorage.setItem("userName", me.name || me.email || "");
    return me;
  } catch {
    return null;
  }
}

```

```
}
```

```
function redirectToLogin() {
```

```
  const next = encodeURIComponent(window.location.pathname + window.location.search);
```

```
  window.location.href = `/login/?next=${next}`;
```

```
}
```

```
async function enforcePageAccess(page) {
```

```
  const publicPages = new Set(["index", "login", "forgot-password", "privacy-policy", "terms"]);
```

```
  if (publicPages.has(page) || page === "artist-dashboard") return true;
```

```
  const me = await getCurrentUser();
```

```
  if (!me) { redirectToLogin(); return false; }
```

```
  return true;
```

```
}
```

```
async function initAuthNav() {
```

```
  const nav = document.getElementById("mainNav");
```

```
  if (!nav) return;
```

```
  const menu = nav.querySelector(".navbar-nav");
```

```
  const actions = nav.querySelector("#navMenu > div.d-flex");
```

```
  const me = await getCurrentUser();
```

```
  if (menu) {
```

```
    menu.innerHTML = me ? `
```

```
      <li class="nav-item"><a class="nav-link" href="/">Home</a></li>
```

```
      <li class="nav-item"><a class="nav-link" href="/artist/">Artists</a></li>
```

```
      <li class="nav-item"><a class="nav-link" href="/gallery/">Gallery</a></li>
```

```
      ${me.role === "client" ? '<li class="nav-item"><a class="nav-link" href="/chat/">Messages</a></li>' : ""}
```

```
      ${me.role === "artist" ? '<li class="nav-item"><a class="nav-link" href="/artist-dashboard/">Dashboard</a></li>' : ""}
```

```
    ` : "";
```

```
  }
```

```

if (actions) {
  if (!me) {
    actions.innerHTML = `
      <a href="/login/?mode=register" class="btn-nav-outline">Register</a>
      <a href="/login/" class="btn-nav-solid">Sign In</a>
    `;
  } else {
    const safeName = escapeHTML(me.name || me.email || "Account");
    actions.innerHTML = `
      <span class="nav-user-name d-none d-lg-inline">${safeName}</span>
      ${me.role === "artist"
        ? '<a href="/artist-dashboard/" class="btn-nav-outline">Dashboard</a>'
        : '<a href="/login/?mode=register&role=artist" class="btn-nav-outline"
id="becomeArtistBtn">Register as Artist</a>'}
      <button type="button" class="btn-nav-solid" id="logoutNavBtn">Logout</button>
    `;
    actions.querySelector("#logoutNavBtn").addEventListener("click", () => {
      clearAuth();
      window.location.href = "/login/";
    });
    actions.querySelector("#becomeArtistBtn").addEventListener("click", event => {
      event.preventDefault();
      clearAuth();
      window.location.href = "/login/?mode=register&role=artist";
    });
  }
}

let ARTISTS = [];
let PORTFOLIO = [];

```

```

async function loadArtists() {
  try {
    const response = await fetch(`${API_BASE}/artists/`);
    if (response.ok) {
      const data = await response.json();
      ARTISTS = data.map(artist => ({
        id: artist.id,
        userId: artist.user_id || artist.user,
        name: artist.user_name,
        location: artist.location,
        style: artist.style,
        priceMin: parseFloat(artist.base_price || 0),
        rating: parseFloat(artist.rating || 0),
        image: artist.avatar_url || artist.avatar || DEFAULT_ARTIST_IMAGE,
        bio: artist.bio || ""
      }));
    } else {
      ARTISTS = [];
    }
  } catch (error) {
    console.error("Error loading artists:", error);
    ARTISTS = [];
  }
}

async function loadPortfolio() {
  try {
    const response = await fetch(`${API_BASE}/artists/portfolio/`);
    PORTFOLIO = response.ok ? await response.json() : [];
  } catch (error) {
    console.error("Error loading portfolio:", error);
  }
}

```

```

    PORTFOLIO = [];
  }
}

function formatINR(n) {
  return "₹" + Number(n).toLocaleString("en-IN");
}

function cap(s) {
  if (!s) return "";
  return s.charAt(0).toUpperCase() + s.slice(1);
}

function escapeHTML(value) {
  return String(value ?? "").replace(/[\&<>"]'/g, char => ({
    "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;", '": "&#39;"
  })[char]);
}

function getQueryParams() {
  const params = {};
  new URLSearchParams(window.location.search).forEach((val, key) => {
    params[key] = val.trim();
  });
  return params;
}

function updateURLParams(filters) {
  const url = new URL(window.location.href);
  Object.entries(filters).forEach(([key, val]) => {
    if (val && val !== "all" && val !== "") url.searchParams.set(key, val);
    else url.searchParams.delete(key);
  });
  window.history.pushState({}, "", url.toString());
}

function getCurrentPage() {

```

```

const parts = window.location.pathname.split("/").filter(Boolean);
return parts[parts.length - 1] || "index";
}

```

```

function buildArtistCard(artist, lazy = true) {
  const profileUrl = `/artist-profile/?id=${artist.id}`;
  return `
    <div class="col-sm-6 col-lg-3 reveal" data-artist-id="${artist.id}">
      <div class="artist-card h-100">
        <div class="artist-img-wrap">
          <span class="art-style-badge">${cap(artist.style)}</span>
          <span class="rating-badge"><i class="bi bi-star-fill"></i>
            ${Number(artist.rating).toFixed(1)}</span>
          <a href="${profileUrl}" aria-label="View ${escapeHTML(artist.name)} profile">
            
          </a>
        </div>
        <div class="artist-body">
          <a href="${profileUrl}" class="artist-name">${escapeHTML(artist.name)}</a>
          <div class="artist-location">
            <i class="bi bi-geo-alt-fill"></i> ${cap(artist.location)}
          </div>
          <a href="${profileUrl}" class="text-brand" style="display:inline-block;font-size:.76rem;margin-top:.45rem;">View Profile</a>
          <div class="artist-meta">
            <div class="artist-price">From ${formatINR(artist.priceMin)}<small> per
            artwork</small></div>
            <a href="/booking/?artist=${encodeURIComponent(artist.name)}&id=${artist.id}"
            class="btn-book">Book Now</a>
          </div>
        </div>
      </div>
    </div>
  `
}

```

```

    </div>`;
}

```

```

function buildGalleryCard(item) {
  const image = item.image_url || item.image || DEFAULT_ARTIST_IMAGE;
  const artistName = item.artist_name || "Unknown Artist";
  const title = item.title || `Artwork by ${artistName}`;
  return `
    <div class="col-sm-6 col-lg-4 reveal">
      <article class="artist-card h-100">
        <div class="artist-img-wrap">
          <span class="art-style-badge">${cap(item.artist_style)}</span>
          
        </div>
        <div class="artist-body">
          <div class="artist-name">${title}</div>
          <div class="artist-location">
            <i class="bi bi-person-fill"></i> ${artistName}
          </div>
          <div class="artist-meta">
            <a href="/artist-profile/?id=${item.artist_id}" class="btn-book">View Artist</a>
          </div>
        </div>
      </article>
    </div>`;
}

```

```

function initNavbar() {
  const nav = document.getElementById("mainNav");
  if (!nav) return;
  window.addEventListener("scroll", () => {
    nav.classList.toggle("scrolled", window.scrollY > 60);
  });
}

```

```
    }, { passive: true });  
}
```

```
function initScrollReveal() {  
  const observer = new IntersectionObserver(entries => {  
    entries.forEach(entry => {  
      if (entry.isIntersecting) {  
        entry.target.classList.add("visible");  
        observer.unobserve(entry.target);  
      }  
    });  
  }, { threshold: 0.1 });  
  document.querySelectorAll(".reveal").forEach(el => observer.observe(el));  
}
```

```
async function initHomePage() {  
  await loadArtists();  
  const grid = document.getElementById("featuredGrid");  
  if (!grid) return;  
  grid.innerHTML = Array(4).fill('<div class="col-sm-6 col-lg-3"><div  
class="skeleton"></div></div>').join("");  
  setTimeout(() => {  
    grid.innerHTML = ARTISTS.slice(0, 4).map(a => buildArtistCard(a, false)).join("");  
    initScrollReveal();  
  }, 600);  
  const searchBtn = document.getElementById("homeSearchBtn");  
  searchBtn?.addEventListener("click", () => {  
    const location = document.getElementById("homeLocation")?.value || "";  
    const style = document.getElementById("homeStyle")?.value || "";  
    const budget = document.getElementById("homeBudget")?.value || "";  
    const params = new URLSearchParams();  
    if (location && location !== "all") params.set("location", location);
```

```

    if (style && style !== "all") params.set("style", style);
    if (budget && budget !== "all") params.set("budget", budget);
    window.location.href = "/artist/" + (params.toString() ? "?" + params.toString() : "");
  });
}

```

```

async function initArtistsPage() {
  await loadArtists();
  const grid = document.getElementById("artistsGrid");
  const filterLoc = document.getElementById("filterLocation");
  const filterStyle = document.getElementById("filterStyle");
  const filterBudget = document.getElementById("filterBudget");
  const filterSearch = document.getElementById("filterSearch");
  const resultsCount = document.getElementById("resultsCount");
  const noResults = document.getElementById("noResults");
  const tagsWrap = document.getElementById("filterTags");
  if (!grid) return;

```

```

function applyFilters() {
  const loc = filterLoc.value.toLowerCase();
  const style = filterStyle.value.toLowerCase();
  const budget = filterBudget.value;
  const search = filterSearch.value.toLowerCase().trim();
  const filtered = ARTISTS.filter(artist => {
    if (loc && loc !== "all" && artist.location !== loc) return false;
    if (style && style !== "all" && artist.style !== style) return false;
    if (budget && budget !== "all") {
      const price = artist.priceMin;
      if (budget === "under1000" && price >= 1000) return false;
      if (budget === "1000-5000" && (price < 1000 || price > 5000)) return false;
      if (budget === "5000-15000" && (price < 5000 || price > 15000)) return false;
      if (budget === "15000-50000" && (price < 15000 || price > 50000)) return false;
    }
  });

```

```

    if (budget === "above50000" && price < 50000) return false;
  }
  if (search && !artist.name.toLowerCase().includes(search)) return false;
  return true;
});
updateURLParams({ location: loc, style, budget, search });

if (!filtered.length) {
  grid.innerHTML = "";
  noResults?.classList.add("show");
} else {
  noResults?.classList.remove("show");
  grid.innerHTML = Array(filtered.length).fill('<div class="col-sm-6 col-lg-3"><div
class="skeleton"></div></div>').join("");
  setTimeout(() => {
    grid.innerHTML = filtered.map(a => buildArtistCard(a)).join("");
    initScrollReveal();
  }, 350);
}

resultsCount && (resultsCount.innerHTML = `Showing <span>${filtered.length}</span>
artist${filtered.length !== 1 ? "s" : ""}`);
if (tagsWrap) renderFilterTags({ loc, style, budget, search });
}

function renderFilterTags(filters) {
  const tags = [];
  if (filters.loc && filters.loc !== "all") tags.push({ key: "location", label: cap(filters.loc) });
  if (filters.style && filters.style !== "all") tags.push({ key: "style", label: cap(filters.style) });
  if (filters.budget && filters.budget !== "all") {
    const labels = {
      under1000: "Under ₹1,000",
      "1000-5000": "₹1K–₹5K",

```

```

    "5000-15000": "₹5K–₹15K",
    "15000-50000": "₹15K–₹50K",
    above50000: "₹50K+"
  };

  tags.push({ key: "budget", label: labels[filters.budget] || filters.budget });
}
if (filters.search) tags.push({ key: "search", label: `${filters.search}` });
tagsWrap.innerHTML = tags.map(t => `<span class="filter-tag" data-
key="${t.key}">${t.label} <span class="remove">×</span></span>`).join("");
}

tagsWrap?.addEventListener("click", e => {
  const tag = e.target.closest(".filter-tag");
  if (!tag) return;
  const key = tag.dataset.key;
  if (key === "location") filterLoc.value = "all";
  if (key === "style") filterStyle.value = "all";
  if (key === "budget") filterBudget.value = "all";
  if (key === "search") filterSearch.value = "";
  applyFilters();
});

[filterLoc, filterStyle, filterBudget].forEach(el => el?.addEventListener("change", applyFilters));
filterSearch?.addEventListener("input", () => {
  clearTimeout(window.__filterTimer);
  window.__filterTimer = setTimeout(applyFilters, 280);
});

const params = getQueryParams();
if (params.location) filterLoc.value = params.location;
if (params.style) filterStyle.value = params.style;
if (params.budget) filterBudget.value = params.budget;

```

```

    if (params.search) filterSearch.value = params.search;
    applyFilters();
  }

  async function initGalleryPage() {
    await loadPortfolio();
    const grid = document.getElementById("galleryGrid");
    const empty = document.getElementById("galleryEmpty");
    if (!grid) return;
    if (!PORTFOLIO.length) {
      grid.innerHTML = "";
      empty?.classList.add("show");
      return;
    }
    empty?.classList.remove("show");
    grid.innerHTML = PORTFOLIO.map(item => buildGalleryCard(item)).join("");
    initScrollReveal();
  }

  async function initBookingPage() {
    await loadArtists();
    const params = getQueryParams();
    const artistId = parseInt(params.id, 10);
    let artist = ARTISTS.find(a => a.id === artistId);
    if (!artist && params.artist) {
      artist = ARTISTS.find(a => a.name.toLowerCase() ===
decodeURIComponent(params.artist).toLowerCase());
    }
    if (!artist) artist = ARTISTS[0];
    artist && populateBooking(artist);
  }

```

```

function populateBooking(artist) {
  const set = (id, val) => { const el = document.getElementById(id); if (el) el.textContent = val; };
  const img = document.getElementById("bookingArtistImg");
  if (img) { img.src = artist.image; img.alt = artist.name; }
  set("bookingArtistName", artist.name);
  set("bookingArtistLocation", cap(artist.location));
  set("bookingArtistStyle", cap(artist.style));
  set("bookingArtistRating", Number(artist.rating).toFixed(1));
  set("bookingArtistBio", artist.bio);

  const fee = Math.round(artist.priceMin * 0.05);
  const base = document.getElementById("basePrice");
  if (base) { base.textContent = formatINR(artist.priceMin); base.dataset.base = artist.priceMin; }
  set("platformFee", formatINR(fee));
  set("totalPrice", formatINR(artist.priceMin + fee));
  document.getElementById("bookingPageTitle")?.textContent = `Book ${artist.name}`;
  document.getElementById("bookingPortfolioLink")?.setAttribute("href", `/artist-profile/?id=${artist.id}#portfolio-heading`);
}

```

```

function initBookingForm() {
  const serviceSelect = document.getElementById("serviceType");
  if (!serviceSelect) return;
  const multipliers = { standard: 1, express: 1.5, premium: 2 };
  serviceSelect.addEventListener("change", () => {
    const m = multipliers[serviceSelect.value] || 1;
    const base = parseInt(document.getElementById("basePrice")?.dataset.base || 0);
    const adj = Math.round(base * m);
    const fee = Math.round(adj * 0.05);
    document.getElementById("basePrice").textContent = formatINR(adj);
    document.getElementById("platformFee").textContent = formatINR(fee);
    document.getElementById("totalPrice").textContent = formatINR(adj + fee);
  });
}

```

```

});

const dateInput = document.getElementById("bookingDate");
if (dateInput) dateInput.min = new Date().toISOString().split("T")[0];

const bookingForm = document.getElementById("bookingForm");
bookingForm?.addEventListener("submit", async e => {
  e.preventDefault();
  const termsAccepted = document.getElementById("agreeTerms")?.checked || false;
  if (!termsAccepted) {
    showSiteNotice({
      type: "error",
      icon: "bi-exclamation-triangle-fill",
      title: "Terms required",
      text: "Please accept the Terms of Service before sending a booking request."
    });
    return;
  }
  const btn = bookingForm.querySelector('[type="submit"]');
  const originalText = btn.textContent;
  const delivery = bookingForm.querySelector('input[name="delivery"]:checked')?.value ||
"digital";
  const deliveryAddress = document.getElementById("deliveryAddress")?.value.trim() || "";

  if (["physical", "both"].includes(delivery) && !deliveryAddress) {
    showSiteNotice({
      type: "error",
      icon: "bi-geo-alt-fill",
      title: "Delivery address required",
      text: "Please enter a delivery address for physical artwork delivery."
    });
    return;
  }

```

```

    }

    btn.textContent = "Confirming...";
    btn.disabled = true;

    const params = getQueryParams();
    const artistId = parseInt(params.id);
    const bookingData = new FormData();
    bookingData.append("artist", artistId);
    bookingData.append("description", document.getElementById("projectDescription")?.value ||
    "");
    bookingData.append("service_type", document.getElementById("serviceType")?.value ||
    "standard");
    bookingData.append("artwork_size", bookingForm.querySelectorAll("select")[1]?.value || "");
    bookingData.append("delivery", delivery);
    bookingData.append("delivery_address", deliveryAddress);
    bookingData.append("preferred_date", document.getElementById("bookingDate")?.value || "");
    bookingData.append("terms_accepted", termsAccepted ? "true" : "");
    const referenceFile = document.getElementById("referenceImage")?.files?.[0];
    if (referenceFile) bookingData.append("reference_image", referenceFile);

    try {
        const response = await authFetch(`${API_BASE}/bookings/create/`, { method: "POST",
        body: bookingData });
        if (response.ok) {
            sessionStorage.setItem("pendingSiteNotice", JSON.stringify({
                type: "success",
                icon: "bi-check-circle-fill",
                title: "Booking request sent",
                text: "The artist will review your commission and reply in Messages."
            }));
            window.location.href = "/";
        }
    }

```

```

    } else {
      const error = await response.json();
      showSiteNotice({
        type: "error",
        icon: "bi-exclamation-triangle-fill",
        title: "Booking not sent",
        text: error.error || error.detail || "Please check the details and try again."
      });
      btn.textContent = originalText;
      btn.disabled = false;
    }
  } catch {
    showSiteNotice({
      type: "error",
      icon: "bi-wifi-off",
      title: "Could not reach Shristi",
      text: "Please try again in a moment."
    });
    btn.textContent = originalText;
    btn.disabled = false;
  }
});
}

function initLoginPage() {
  // login/register tab switching, validation, and request logic
}

async function initArtistProfilePage() {
  // loads artist detail and populates profile sections
}

```

```

async function initChatPage() {
  // manages chat inbox, thread messages, file uploads, polling
}

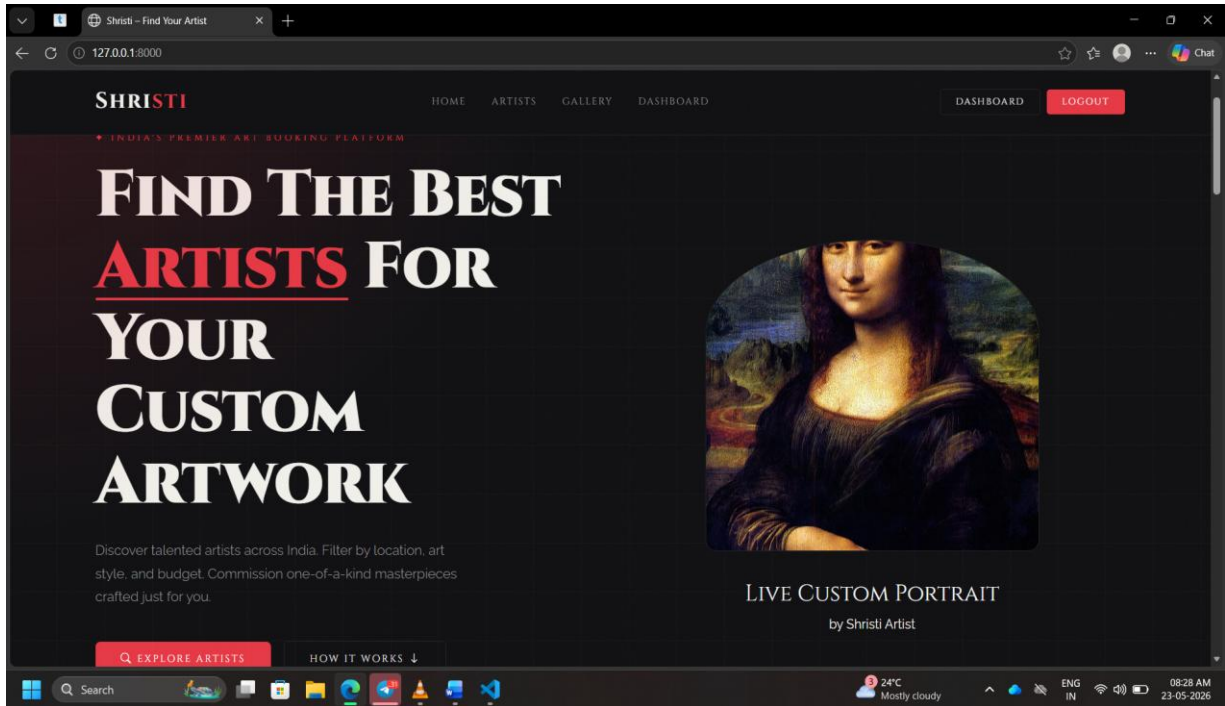
function initSmoothScroll() {
  document.querySelectorAll('a[href$="#howItWorks"]').forEach(link => {
    link.addEventListener("click", function (e) {
      const page = getCurrentPage();
      if (["index", "index.html", ""].includes(page)) {
        e.preventDefault();
        document.getElementById("howItWorks")?.scrollIntoView({ behavior: "smooth", block:
"start" });
      }
    });
  });
  document.querySelectorAll('a[href^="#"]').forEach(anchor => {
    anchor.addEventListener("click", function (e) {
      const targetId = this.getAttribute("href").substring(1);
      const el = document.getElementById(targetId);
      if (el) { e.preventDefault(); el.scrollIntoView({ behavior: "smooth" }); }
    });
  });
  document.addEventListener("DOMContentLoaded", async () => {
    initNavbar();
    await initAuthNav();
    showPendingSiteNotice();
    initScrollReveal();
    initSmoothScroll();
    const page = getCurrentPage();
    const canAccessPage = await enforcePageAccess(page);
    if (!canAccessPage) return;
    initCustomerMessageNotice();
  });
}

```

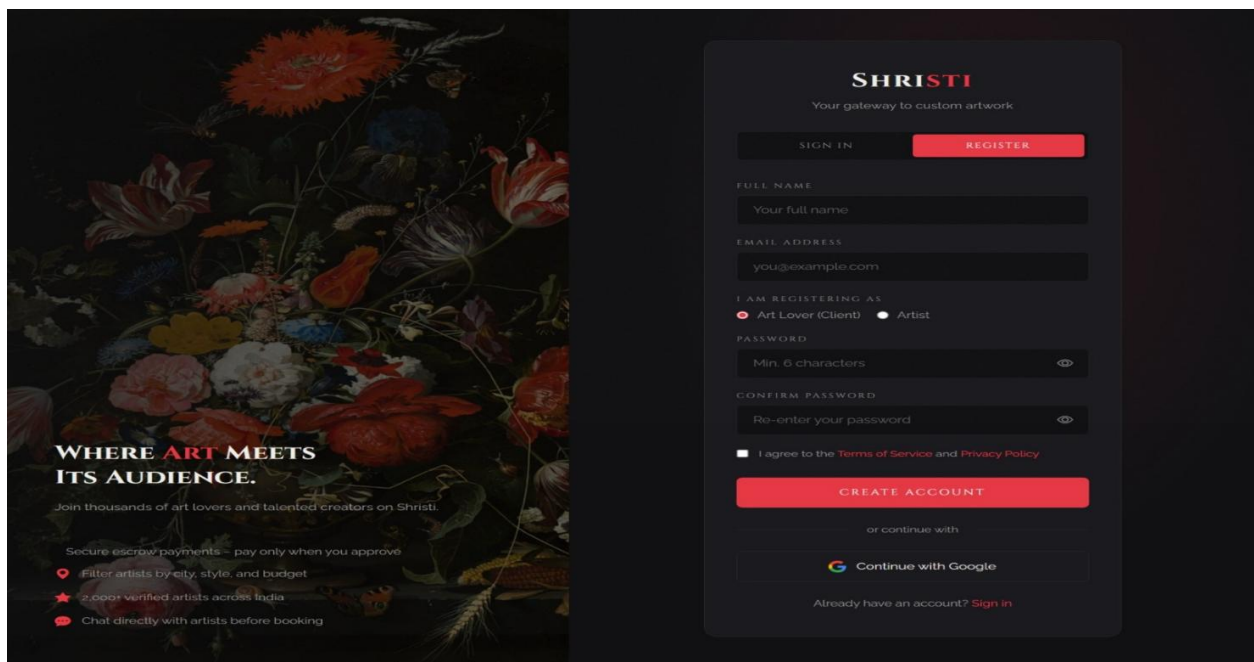
```
initCustomerBookingNotice();
setInterval(() => {
    initCustomerMessageNotice();
    initCustomerBookingNotice();
}, 8000);
if (["index", "index.html", ""].includes(page)) await initHomePage();
if (["artist", "artist.html"].includes(page)) await initArtistsPage();
if (["gallery", "gallery.html"].includes(page)) await initGalleryPage();
if (["booking", "booking.html"].includes(page)) { await initBookingPage(); initBookingForm();
}
if (["login", "login.html"].includes(page)) initLoginPage();
if (["artist-profile", "artist-profile.html"].includes(page)) await initArtistProfilePage();
if (["chat", "chat.html"].includes(page)) await initChatPage();
});
```

11. SCREENSHOTS

HOME PAGE



REGISTER PAGE



SIGN IN PAGE



WHERE **ART** MEETS ITS AUDIENCE.

Join thousands of art lovers and talented creators on Shristi.

Secure escrow payments - pay only when you approve.

- 📍 Filter artists by city, style, and budget
- ★ 2,000+ verified artists across India
- 💬 Chat directly with artists before booking

SHRISTI

Your gateway to custom artwork

[SIGN IN](#)[REGISTER](#)

EMAIL ADDRESS

you@example.com

PASSWORD

Enter your password

👁

[Forgot password?](#)

I AM SIGNING IN AS

☒ Art Lover ☐ Artist

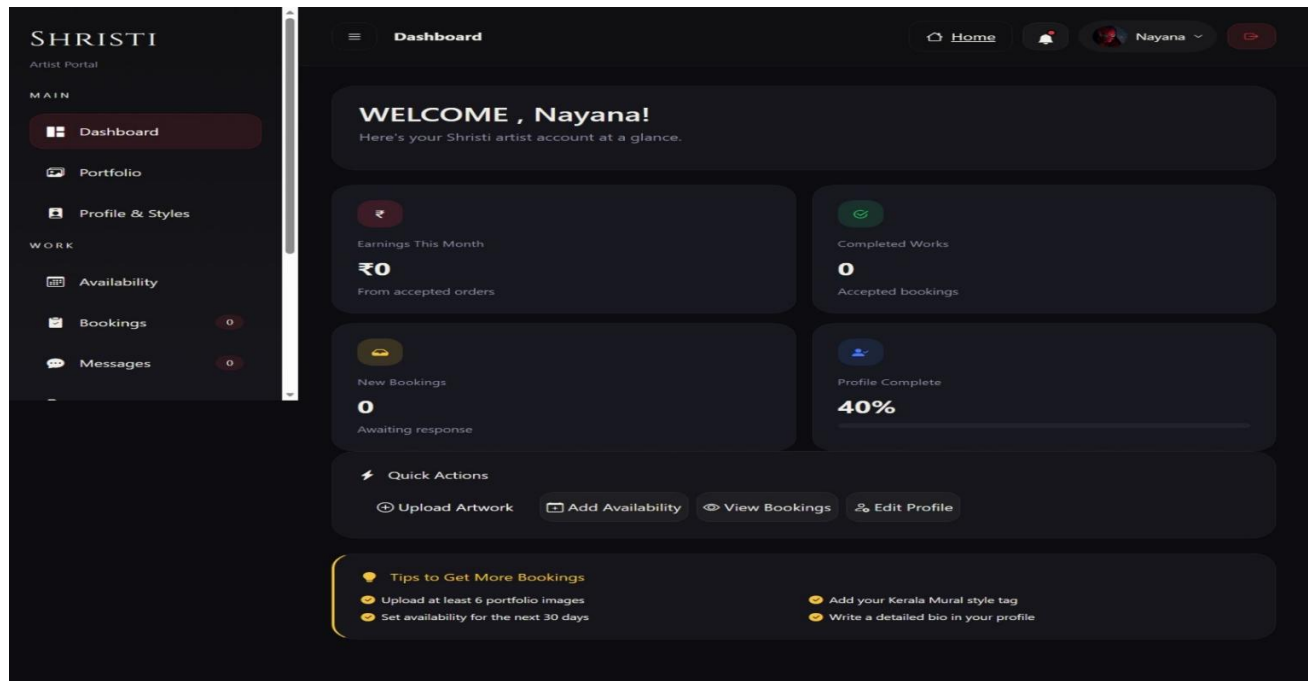
[SIGN IN](#)

or continue with

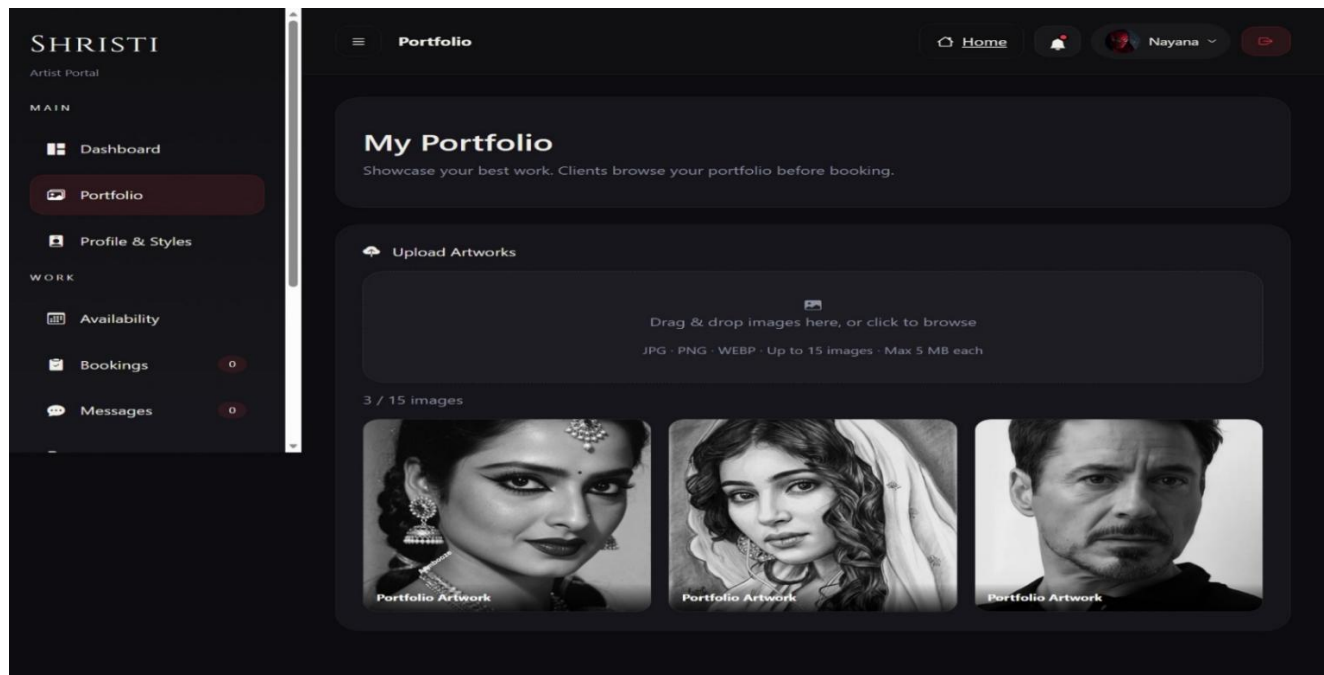
 Continue with Google

Don't have an account? [Register here](#)

ARTIST DASHBOARD



PORTFOLIO



ARTIST PROFILE AND STYLES

SHRISTI

Artist Portal

MAIN

Dashboard

Portfolio

Profile & Styles

WORK

Availability

Bookings0

Messages0

Profile & Styles

Home

🔔

 Nayana

🔖

Profile & Art Styles

Your public profile — what clients see when they find you.

Profile Photo



Upload a professional photo
Square image, at least 300×300 px. Max 4 MB. JPG or PNG.

Choose Photo

Basic Information

Full Name

Nayana

Phone Number

9048967146

City / Location

Kochi (Cochin)

Years of Experience

1–3 years

Bio / Artist Statement

portrait Artist

Mediums You Work With

Acrylic, digital

Comma-separated list

Art Styles

Add your speciality styles. Clients use these to filter and discover you.

Your Style Tags

digital

sketch

Type a style then press Add or Enter...

+ Add

Suggested Styles (click to add)

portrait abstract watercolour oil sketch digital mandala mural

Reset

Save Profile

PRICING AND SERVICES

Dashboard

Portfolio

Profile & Styles

WORK

Availability

Bookings0

Messages0

Pricing & Services

ACCOUNT

Back to Home

Home

🔔

Nayana

⚙️

Pricing & Services

Set your prices. Clients see these on your public profile.

₹ Size-Based Pricing

Small (A4 / 8×10 in)

6000

₹ per piece

Medium (A3 / 12×16 in)

e.g. 3000

₹ per piece

Large (A2 / 18×24 in)

e.g. 6000

₹ per piece

Custom / Mural (per sq.ft)

e.g. 800

₹ / sq.ft

📦 Add-on Services

Framing (add-on)

e.g. 500

Gift Wrapping (add-on)

e.g. 200

Express Delivery Surcharge

e.g. 1000

Digital File Price

e.g. 500

🕒 Turnaround Time

Standard Turnaround

Select...▼

Rush Order Turnaround

Select...▼

↺ Reset

💾 Save Pricing

BROWSE ARTISTS

SHRISTI

HOMEARTISTSGALLERYMESSAGES

REGISTER AS ARTISTLOGOUT

Home > Artists

DISCOVER & BOOK

BROWSE ARTISTS

Find the perfect artist for your custom artwork across India.

SEARCH BY NAME

LOCATION

ART STYLE

BUDGET

e.g. Rohan, Priya...

All Locations

All Styles

Any Budget

SHOWING 12 ARTISTS

DIGITAL

NIHALFAZI02@GMAIL.COM

Kerala

View Profile

FROM ₹1,500

per artwork

BOOK NOW

PORTRAIT

NAYANENDHU CU

Bengaluru

View Profile

FROM ₹2,000

per artwork

BOOK NOW

OIL

ATHIRU

Kerala

View Profile

FROM ₹4,997

per artwork

BOOK NOW

PORTRAIT

NIHAL

Kerala

View Profile

FROM ₹1,000

per artwork

BOOK NOW

MANDALA

AIZA MEHWIN

Kerala

View Profile

FROM ₹0

per artwork

BOOK NOW

DIGITAL

ANAGHA

Jaipur

View Profile

FROM ₹1,500

per artwork

BOOK NOW

DIGITAL

FIDHU

Kerala

View Profile

FROM ₹0

per artwork

BOOK NOW

SKETCH

NIYA

Mumbai

View Profile

FROM ₹1,999

per artwork

BOOK NOW

DIGITAL

ANAMIKA

Kerala

View Profile

FROM ₹500

per artwork

BOOK NOW

DIGITAL

NAYANA

Kerala

View Profile

FROM ₹6,000

per artwork

BOOK NOW

DIGITAL

NIHAL

Kerala

View Profile

FROM ₹0

per artwork

BOOK NOW

SHRISTI

India's premier platform for discovering and booking talented artists for custom artwork.

PLATFORM

Browse Artists

Dashboard

Commissions

ART STYLES

Portrait

Abstract

Watercolour

Mandala

Digital

SUPPORT

Contact Us

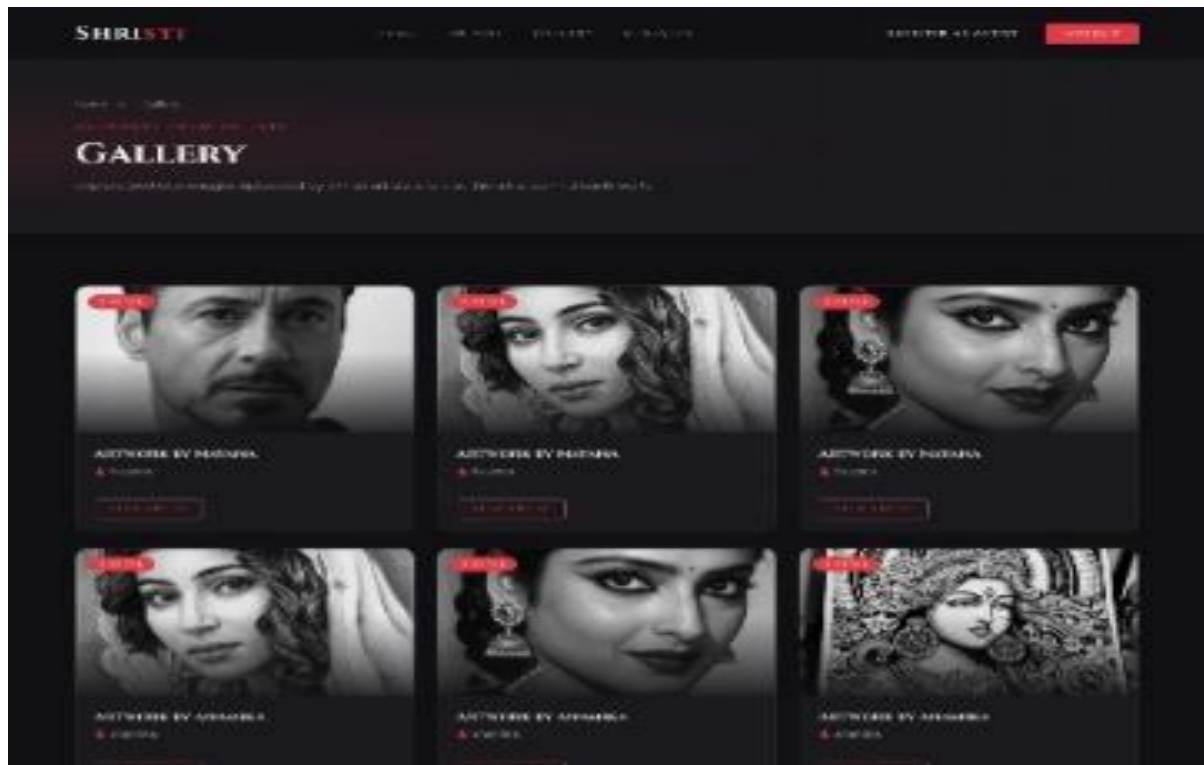
Privacy Policy

Terms

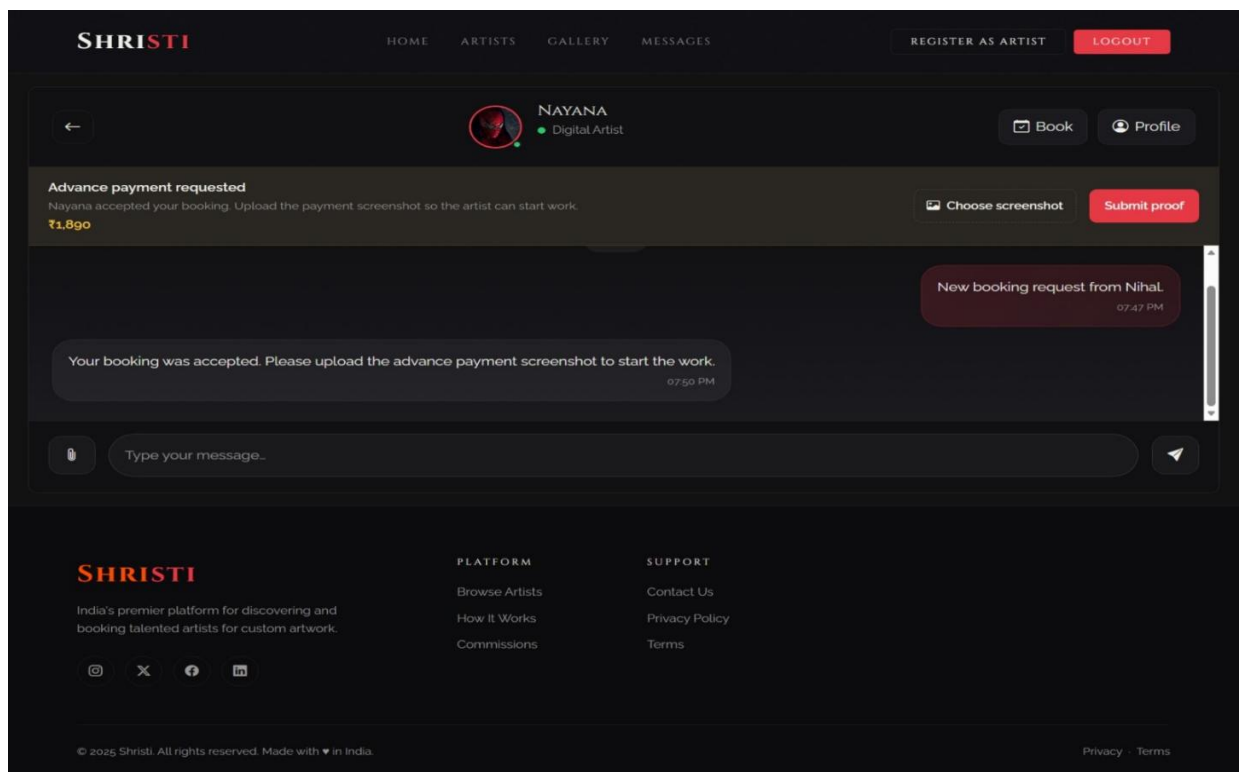
© 2025 Shristi. All rights reserved.

PrivacyTerms

GALLERY



MESSAGES



BOOKING PAGE

SHRISTI

HOMEARTISTSGALLERYMESSAGES

REGISTER AS ARTISTLOGOUT

Home > Artists > Book

COMMISSION ARTWORK

BOOK YOUR ARTIST

Fill in the details below and we'll confirm your booking within 24 hours.



NAYANA

Kerala

Digital

★ 0.0 Rating

portrait Artist

VIEW ARTIST PORTFOLIO

Base Price

₹6,000

Platform Fee (5%)

₹300

TOTAL

₹6,300

← Choose a different artist

STEP 1 OF 1 - COMMISSION DETAILS

FILL YOUR BOOKING DETAILS

YOUR FULL NAME *

Nihal

EMAIL ADDRESS *

nihal

PHONE NUMBER *

75948 01686

SERVICE TYPE *

Standard (1x price)

PREFERRED START DATE *

mm/dd/yyyy

ARTWORK SIZE

A4 (21 x 29 cm)

DELIVERY ADDRESS (REQUIRED FOR PHYSICAL DELIVERY)

Kannanjeri(H), Malyekkal, kerala

PROJECT DESCRIPTION *

High quality portrait

REFERENCE IMAGE (OPTIONAL)

Choose File

875fd2e18168e6cce21fb592121d4b42.jpg

JPG or PNG up to 10 MB.

DELIVERY PREFERENCE

☐ Digital File

☐ Physical Delivery

☒ Both

☒ I agree to Shristi's Terms of Service and understand that payment is held in escrow until I approve the final artwork.

SEND BOOKING REQUEST

Advance payment is requested only after the artist accepts your booking.

SHRISTI

India's premier platform for discovering and booking talented artists for custom artwork.

Instagram

X

Facebook

NAVIGATE

Home

Artists

Book

Login

SUPPORT

Contact Us

Privacy Policy

Terms

© 2025 Shristi. All rights reserved.

Privacy · Terms

93 | Page

ORDER VIEW

The screenshot shows the 'ORDER VIEW' page in the SHRISTI Artist Portal. The left sidebar contains navigation links: 'MAIN' (Dashboard, Portfolio, Profile & Styles) and 'WORK' (Availability, Bookings, Messages). The main content area displays order details for a customer named Nihal. Key information includes: Total amount of ₹6300.00, artwork size of A4 (21 x 29 cm), delivery address in Kannanjeri(H), Kerala, and a submitted payment. The project is described as a 'High quality portrait'. A reference image of a woman's face is shown. A payment screenshot from a UPI transaction is displayed, showing a payment of ₹500 on 20 Jun 2020. The status is 'accepted'. At the bottom, there are links to 'Message Customer' and a green button to 'Verify Payment & Start Work'.

SHRISTI
Artist Portal

MAIN

- Dashboard
- Portfolio
- Profile & Styles

WORK

- Availability
- Bookings 0
- Messages 0

Customer: Nihal

Total: ₹6300.00

Artwork size: A4 (21 x 29 cm)

Delivery address: Kannanjeri(H), Maliyekkal, kerala

Payment: submitted

Start date: 2026-05-23

Advance: ₹1890.00

Delivery: both

Status: accepted

Project description
High quality portrait

Reference image

Payment screenshot

✓ Paid ₹ 500
20 Jun 2020 • 1:06 pm

UPI Transaction ID
017213727379

To
**** 9wce

From: DEBASISH DASH (State Bank of India)
debashishdash50.dd@sbki

[Message Customer](#) [Verify Payment & Start Work](#)

Accepted

Close

FORGOT PASSWORD

The screenshot shows the 'FORGOT PASSWORD' form in the SHRISTI application. The form is centered on a dark background. It features the SHRISTI logo and tagline 'Your gateway to custom artwork'. Below this, it says 'RESET PASSWORD' and 'We'll send a reset link to your email'. There is a text input field for the email address, which currently contains 'you@example.com'. Below the input field is an orange button labeled 'Send Reset Link'. At the bottom of the form, there is a link that says '← Back to Sign In'.

SHRISTI
Your gateway to custom artwork

RESET PASSWORD
We'll send a reset link to your email

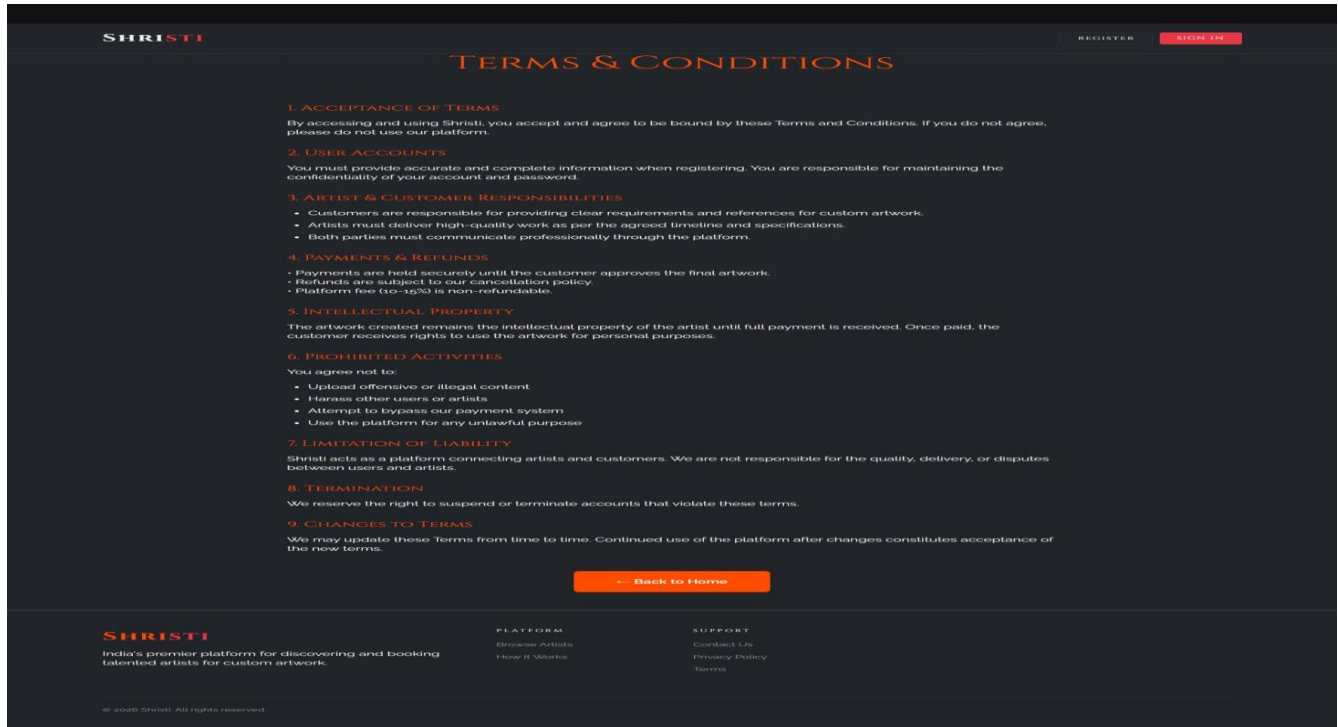
EMAIL ADDRESS

you@example.com

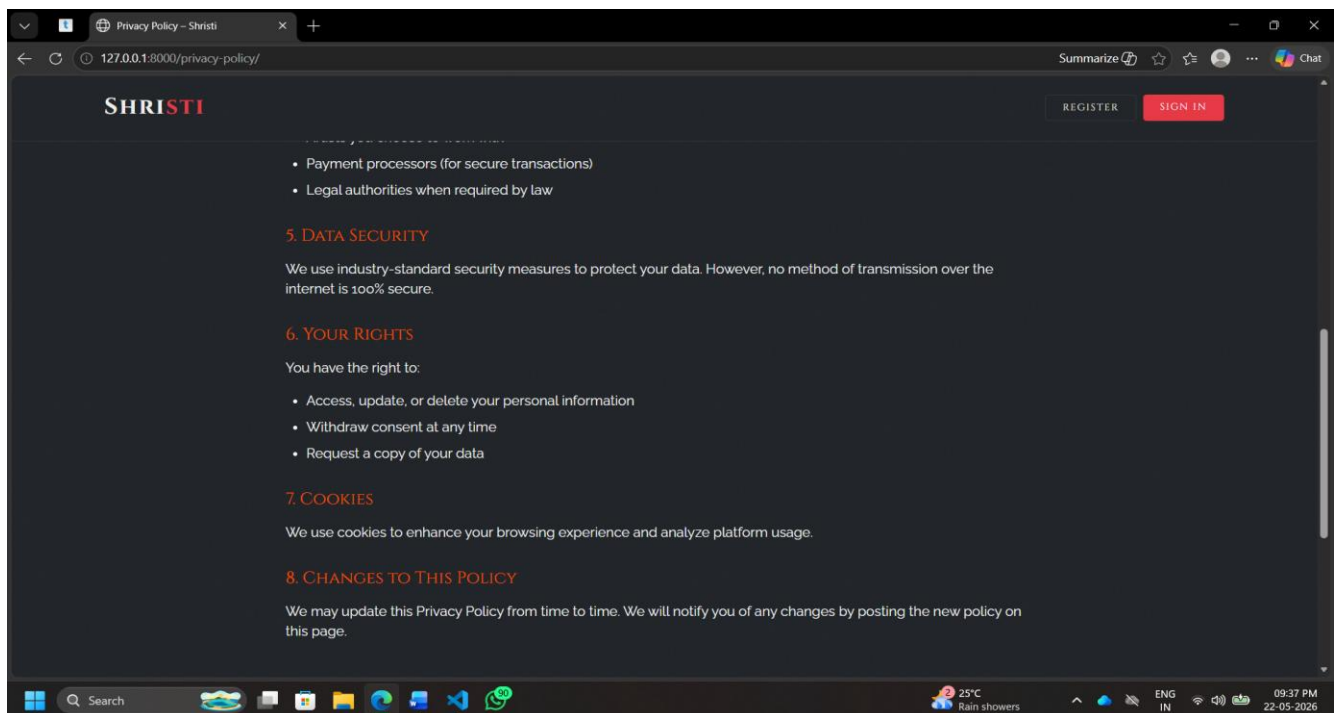
[Send Reset Link](#)

[← Back to Sign In](#)

TERMS AND CONDITIONS



PRIVACY POLICY



12.CONCLUSION

The Shristi platform successfully demonstrates a full-featured, end-to-end artist booking application designed for modern web usage. It combines a Django backend with a clean static frontend to deliver a polished marketplace experience for artists and clients.

1. What the Project Achieves

- **Artist discovery and profiles:** Users can browse artists by style, location, and budget, view curated portfolios, and explore detailed artist profiles.
- **Commission booking workflow:** Clients can request custom artwork, select service options, provide delivery details, and submit booking forms through a streamlined interface.
- **User authentication and role-aware navigation:** The system supports login, registration, session persistence, and conditional navigation for clients and artists.
- **Real-time-like messaging flow:** The chat interface allows clients and artists to exchange messages, including image attachments and threaded conversations.

2. Architecture and Implementation

- **Backend:** Django serves as the project's core, managing models, serializers, endpoints, and page routing. The project structure is organized into reusable apps such as artists, [bookings](#), messaging, and users.
- **Frontend:** Static assets like [style.css](#) and [main.js](#) provide the UI and interactive behavior. The CSS uses custom properties for theme consistency, while JavaScript handles state, navigation, filtering, and API communication.
- **API-driven UI:** Frontend pages rely on API calls for data such as artist listings, portfolios, user details, messages, and bookings. This creates a responsive user experience and separates presentation from backend logic.

3. Design and UX Highlights

- **Consistent branding:** The site uses a dark, modern visual palette with red accents, custom typography, and polished card-based layouts.
- **Reusable UI components:** Buttons, cards, badges, reveal animations, and alert notices are defined once and reused across pages.

- **Responsive and accessible layout:** The design supports mobile-friendly behavior, sticky navigation, smooth scrolling, and clear feedback for user actions.
- **User-friendly filters:** The artist discovery page provides location, style, budget, and search filters with live result updating and tag-based filter removal.

4. Strengths and Readiness

- **Modular codebase:** The app's separation into Django apps and frontend utility functions makes it easy to maintain and extend.
- **Strong proof of concept:** The core business flow — from browsing artists to booking and messaging — is complete and coherent.
- **Practical expansion points:** The project is ready for future improvements like payment processing, notification emails, dashboard analytics, admin moderation, and enhanced security.

5. Final Assessment

Shristi is more than a simple demo: it is a viable marketplace MVP that balances backend robustness with user-facing polish. It proves that a specialized artist booking service can be built effectively using Django as the server foundation and focused static frontend scripting for a modern interactive experience.

In summary, Shristi provides a solid, scalable foundation for a creative services platform, offering real-world booking flows, artist-client communication, and a visually consistent interface that supports both brand identity and functional growth.

13. FUTURE ENHANCEMENTS

Payment integration: Add secure payment processing so clients can pay artists directly through the platform instead of only uploading proof screenshots.

Notification system: Implement email and in-app notifications for booking updates, new messages, payment requests, and artist responses.

Artist dashboard improvements: Expand artist dashboard features with booking management, portfolio editing, service packages, and availability controls.

Admin panel: Add an administrative interface for moderating artists, managing bookings, reviewing messages, and handling disputes.

Search and recommendation: Improve artist discovery with relevance-based search, sorting options, recommended artists, and user favorites.

Reviews and ratings: Add client reviews and rating systems for artists, increasing trust and helping users choose quality creators.

Performance optimization: Introduce server-side pagination, caching, and asset bundling to speed up page load times and API responses.

Mobile-first UX: Further optimize the interface for mobile devices with improved navigation, touch-friendly controls, and responsive form layouts.

These enhancements would turn Shristi from a strong MVP into a more complete marketplace platform with better monetization, moderation, and user engagement.

14. BIBLIOGRAPHY

WEB REFERENCE

- Google Fonts

[`https://fonts.googleapis.com/css2?family=Cinzel:wght@400;600;700;900&family=Raleway:wght@300;400;500;600&display=swap`](https://fonts.googleapis.com/css2?family=Cinzel:wght@400;600;700;900&family=Raleway:wght@300;400;500;600&display=swap)

- Bootstrap 5

[- `https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css`](https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css)

[- `https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js`](https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js)

- Bootstrap Icons

[- `https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css`](https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css)

<https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css>

- Placeholder avatars

[- `https://i.pravatar.cc/400?img=47`](https://i.pravatar.cc/400?img=47)

[- `https://i.pravatar.cc/80?img=47`](https://i.pravatar.cc/80?img=47)

[- `https://i.pravatar.cc/200?img=47`](https://i.pravatar.cc/200?img=47)

- Example portfolio / hero images

[- `https://images.unsplash.com/photo-1579783902614-a3fb3927b6a5?w=900&q=80`](https://images.unsplash.com/photo-1579783902614-a3fb3927b6a5?w=900&q=80)

[- `https://images.unsplash.com/photo-1489424731084-a5d8b219a5bb?w=400&q=80`](https://images.unsplash.com/photo-1489424731084-a5d8b219a5bb?w=400&q=80)

- plus several other `https://images.unsplash.com/...` image URLs used in templates

- Django documentation references

[- `https://docs.djangoproject.com/en/6.0/topics/settings/`](https://docs.djangoproject.com/en/6.0/topics/settings/)

[- `https://docs.djangoproject.com/en/6.0/ref/settings/`](https://docs.djangoproject.com/en/6.0/ref/settings/)

[- `https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/`](https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/)

[- `https://docs.djangoproject.com/en/6.0/ref/settings/#databases`](https://docs.djangoproject.com/en/6.0/ref/settings/#databases)

[- `https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators`](https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators)

- <https://docs.djangoproject.com/en/6.0/topics/i18n/>
- <https://docs.djangoproject.com/en/6.0/howto/static-files/>
- <https://docs.djangoproject.com/en/6.0/howto/deployment/asgi/>
- <https://docs.djangoproject.com/en/6.0/howto/deployment/wsgi/>

> These references cover the frontend assets, placeholder image sources, and Django documentation links actually found in the project files.

BOOKS

- **Two Scoops of Django — Audrey Roy Greenfeld & Daniel Roy Greenfeld (2nd ed.):** Practical Django patterns and best practices for structuring projects and apps.
- **Django for Professionals — William S. Vincent:** Production-ready Django topics (deployments, security, REST APIs).
- **Designing Data-Intensive Applications — Martin Kleppmann:** Principles of data systems, storage, replication, and scalability relevant to backend design.
- **REST in Practice — Jim Webber, Savas Parastatidis & Ian Robinson:** Practical guidance on designing robust RESTful APIs.
- **Eloquent JavaScript — Marijn Haverbeke:** Modern JavaScript fundamentals and idiomatic usage for frontend logic.
- **You Don't Know JS (series) — Kyle Simpson:** Deep dives into core JavaScript behavior and advanced patterns.
- **CSS: The Definitive Guide — Eric A. Meyer & Estelle Weyl:** Comprehensive reference for CSS layout, selectors, and modern styling techniques.
- **Don't Make Me Think — Steve Krug:** Usability and UX principles to improve site navigation and user flows.
- **Refactoring — Martin Fowler:** Techniques for improving code structure and maintainability without changing behavior.
- **The Pragmatic Programmer — Andrew Hunt & David Thomas:** Software engineering practices, tooling, and professional workflows.
- **The Web Application Hacker's Handbook — Dafydd Stuttard & Marcus Pinto:** Important security considerations and common web vulnerabilities to guard against.